

Java 24 — Vector API (Ninth Incubator)

Property	Details
Java Version	24
Year	2025
Feature	Vector API (Ninth Incubator)
JEP	JEP 469
UniqueClassName	VectorAPI
Status	Ninth Incubator
Package	jdk.incubator.vector
Primary purpose	Express SIMD/vector computations in Java so the JIT can compile them to appropriate CPU vector instructions
Main benefit	Process multiple primitive values with a single vector instruction where hardware supports it
Typical workloads	Numeric computing, image/audio processing, signal processing, scientific computing, ML, cryptography, compression
Important	This is an incubator API , not a standard permanent Java SE API

The Vector API is designed to allow Java applications to express vector computations while remaining portable across CPU architectures. The JIT can map the operations to SIMD instructions available on the target processor. JEP 469 is the ninth incubator iteration and was intended to continue improving the API based on experience with previous incubations.

1. What Is the Vector API?

Normally, code such as:

```
for (int i = 0; i < a.length; i++) {  
    result[i] = a[i] + b[i];  
}
```

processes values conceptually one at a time:

```
a[0] + b[0]  
a[1] + b[1]
```

```
a[2] + b[2]
a[3] + b[3]
```

Vector/SIMD processing can operate on multiple values together:

```
a[0] a[1] a[2] a[3]
+    +    +    +
b[0] b[1] b[2] b[3]
-----
r[0] r[1] r[2] r[3]
```

The CPU may execute this using a SIMD/vector instruction.

Java's Vector API provides abstractions such as:

```
IntVector
FloatVector
DoubleVector
LongVector
ByteVector
ShortVector
```

and vector species describing the supported vector shape.

2. Simple Example

A simplified Java 24 Vector API example:

```
import jdk.incubator.vector.*;

static void add(float[] a, float[] b, float[] result) {

    VectorSpecies<Float> SPECIES = FloatVector.SPECIES_PREFERRED;

    int i = 0;

    for (; i < SPECIES.loopBound(a.length); i += SPECIES.length()) {

        var va = FloatVector.fromArray(SPECIES, a, i);
        var vb = FloatVector.fromArray(SPECIES, b, i);

        var vr = va.add(vb);

        vr.intoArray(result, i);
    }

    for (; i < a.length; i++) {
```

```

    result[i] = a[i] + b[i];
  }
}

```

The important part is:

```
var vr = va.add(vb);
```

Instead of expressing only scalar addition, you express **vector addition**.

3. Advantages and Disadvantages

Advantages	Disadvantages
Can exploit SIMD CPU instructions	API is still incubating
Can process multiple primitive values simultaneously	More complex than ordinary loops
Potentially significant performance improvement for suitable workloads	Performance improvement is workload and hardware dependent
Explicitly expresses vectorizable algorithms	Requires understanding of vector lanes/species
Portable across supported CPU architectures	Exact generated instructions depend on JVM and CPU
Can work with masks	Mask handling adds complexity
Supports multiple primitive numeric types	Not every Java type/operation is supported
Can reduce per-element loop overhead	Poorly structured code may see little benefit
Useful for compute-intensive workloads	Usually unnecessary for ordinary business logic
Works with JIT optimization	Requires careful benchmarking
Can avoid writing architecture-specific intrinsics	Still requires low-level performance knowledge
Can complement JIT auto-vectorization	Manual vector code increases maintenance burden
Suitable for arrays and numerical algorithms	Irregular memory access reduces benefits

4. Why Use Vector API Instead of a Normal Loop?

Consider:

```
for (int i = 0; i < data.length; i++) {  
    data[i] *= 2;  
}
```

For a sufficiently large array, the JVM may already optimize a simple loop through **auto-vectorization**.

This is why you should not automatically rewrite every loop using Vector API.

Vector API becomes particularly interesting when:

- the algorithm is difficult for the JIT to auto-vectorize,
- you need explicit control over vector operations,
- the workload is dominated by numeric computation,
- performance profiling identifies the loop as a hotspot.

5. When to Use

Situation	Use Vector API?	Reason
Large numerical arrays	✓ Excellent candidate	Many independent operations
Matrix/vector calculations	✓ Excellent	Highly parallel arithmetic
Image processing	✓ Excellent	Pixels can often be processed independently
Audio processing	✓ Excellent	Samples can be processed in batches
Signal processing	✓ Excellent	SIMD-friendly mathematical operations
Scientific computing	✓ Excellent	Large amounts of numeric computation
Machine-learning kernels	✓ Good	Many operations are SIMD-friendly
Compression/decompress	✓ Good	Some algorithms have vectorizable

sion		stages
Cryptographic primitives	✓ Potentially	Some operations benefit from SIMD
Database numeric processing	□ Sometimes	Depends on the hot path
Financial calculations	□ Sometimes	Depends on algorithm/data layout
Large array transformations	✓ Good	Independent element operations
Simple CRUD application	✗ Usually no	Little benefit
I/O-bound application	✗ Usually no	CPU isn't the bottleneck
Small arrays	✗ Usually no	Vectorization overhead may dominate

6. When NOT to Use

Situation	Why Avoid It?	Better Choice
Simple business logic	Unnecessary complexity	Ordinary Java
CRUD application	Usually database/I/O bound	Normal Java
REST controller logic	Vectorization rarely helps	Normal Java
Small collections	Too little work to amortize overhead	Normal loop
Heavy object processing	Vector API works primarily with primitive numeric vectors	Ordinary objects/Streams
Pointer/reference-heavy algorithms	Poor SIMD fit	Normal Java
Branch-heavy algorithms	SIMD may be inefficient	Scalar code
Irregular memory access	Poor vector utilization	Conventional loop/data structure
Performance isn't a problem	Optimization is premature	Simple implementation

Team lacks low-level performance expertise	Maintenance risk	Normal Java
Portability across old Java versions is critical	Incubator API dependency	Standard Java
Code changes rarely justify benchmarking	Hard to prove benefit	Keep simple implementation

7. Real-Time Use Case — Image Processing

Suppose an image has millions of pixel values:

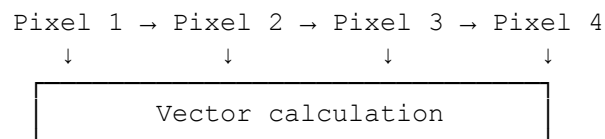
```
R G B R G B R G B ...
```

A grayscale operation might calculate:

```
gray = 0.299R + 0.587G + 0.114B
```

For millions of pixels, processing each pixel independently is SIMD-friendly.

Conceptually:



Vector API can process several numeric values together.

Suitable for

- Image filters
 - Color conversion
 - Brightness adjustment
 - Image resizing kernels
 - Pixel normalization
-

8. Real-Time Use Case — Audio Processing

Audio data often consists of large arrays of samples:

sample1 sample2 sample3 sample4 ...

Suppose every sample needs scaling:

`sample[i] = sample[i] * volume;`

A Vector API implementation can process multiple samples simultaneously.

Conceptually:

```
Scalar:
s1 × volume
s2 × volume
s3 × volume
s4 × volume
```

```
Vector:
[s1 s2 s3 s4] × volume
```

Useful for:

- Audio mixing
- Volume adjustment
- Equalizers
- DSP algorithms
- Noise processing
- Audio normalization

9. Real-Time Use Case — Machine Learning

Many ML operations contain loops like:

```
for (int i = 0; i < n; i++) {
    output[i] =
        input[i] * weight[i] + bias[i];
}
```

This is a classic SIMD candidate.

Conceptually:

```
Input vector
[ x1 x2 x3 x4 ]
```

```
Weight vector
[ w1 w2 w3 w4 ]
```

×

[x1*w1 x2*w2 x3*w3 x4*w4]

Potential applications:

- Feature normalization
- Dot products
- Activation functions
- Quantized arithmetic
- Neural-network kernels

For serious ML workloads, however, specialized libraries and accelerators such as GPUs may still be more appropriate.

10. Real-Time Use Case — Financial Analytics

Suppose millions of prices need normalization:

```
for (int i = 0; i < prices.length; i++) {  
    normalized[i] =  
        (prices[i] - mean) / standardDeviation;  
}
```

This has independent operations for each element.

Vectorization can potentially process multiple prices per CPU instruction.

Useful for:

- Risk calculations
- Portfolio analytics
- Monte Carlo calculations
- Pricing models
- Time-series normalization

But benchmark carefully because financial algorithms may contain branches and dependencies that reduce SIMD benefits.

11. Real-Time Use Case — Database/ETL Processing

Suppose an ETL pipeline needs to transform millions of primitive values:

```
for (int i = 0; i < values.length; i++) {
```

```
    values[i] = values[i] * 100 + 5;
}
```

This is a good SIMD candidate because:

```
value[i]
  ↓
independent calculation
  ↓
value[i]
```

There are no dependencies between iterations.

Vector API can potentially improve throughput.

12. Real-Time Use Case — Signal Processing

Suppose a signal contains:

```
x[0], x[1], x[2], ..., x[n]
```

and you need:

```
y[i] = x[i] * coefficient
```

This is naturally vectorizable.

Applications include:

- Radar
 - Telecommunications
 - Medical signal processing
 - Sensor processing
 - FFT-related operations
 - Digital signal processing
-

13. Real-Time Use Case — Vector Dot Product

Consider:

```
static float dot(float[] a, float[] b) {
    float sum = 0;
```

```

    for (int i = 0; i < a.length; i++) {
        sum += a[i] * b[i];
    }

    return sum;
}

```

The fundamental operation:

$$a[i] \times b[i]$$

is highly suitable for SIMD.

Conceptually:

```

A = [a1 a2 a3 a4]
B = [b1 b2 b3 b4]

```

$$A \times B$$

$$= [a1b1 \ a2b2 \ a3b3 \ a4b4]$$

then horizontal reduction

$$\begin{array}{c} \downarrow \\ a1b1 + a2b2 + a3b3 + a4b4 \end{array}$$

This type of operation occurs in:

- ML
- Scientific computing
- Search/ranking algorithms
- Signal processing
- Numerical analytics

14. Vector API and Masks

Vector API supports masks for conditional vector operations.

Conceptually:

Values:

```
[10 20 30 40]
```

Mask:

```
[ T  F  T  F ]
```

Operation applies to:

[10 30]

For example, you may want:

```
if value > threshold:
    value = value * 2
```

A vector mask can represent the condition for each lane.

This is useful for:

- Conditional numeric transformations
- Image thresholding
- Filtering
- Clamping
- Scientific calculations

15. Vector API vs Parallel Streams

These are **not equivalent technologies**.

Aspect	Vector API	Parallel Stream
Parallelism type	SIMD/data-level	Thread-level
Executes multiple values per CPU instruction	✓	Not directly
Uses multiple CPU cores	Not necessarily	✓
Primary purpose	Data parallelism	Task/data parallelism
Best for	Numeric operations	Independent high-level operations
Primitive arrays	★ ★ ★ ★ ★	★ ★ ★
Object processing	★	★ ★ ★ ★
Ease of use	★ ★	★ ★ ★ ★ ★
Low-level control	★ ★ ★ ★ ★	★ ★
Typical overhead	Low once optimized	Higher
Hardware awareness	CPU SIMD	CPU cores/threads

Requires careful benchmarking	✓	✓

You can potentially combine techniques, but they solve different problems.

16. Vector API vs Ordinary Loop

Aspect	Ordinary Loop	Vector API
Simplicity	★ ★ ★ ★ ★	★ ★ ★
Maintainability	★ ★ ★ ★ ★	★ ★ ★
Portability	★ ★ ★ ★ ★	★ ★ ★ ★
Explicit SIMD	★ ★	★ ★ ★ ★ ★
Potential performance	★ ★ ★ ★	★ ★ ★ ★ ★
Control	★ ★	★ ★ ★ ★ ★
Learning curve	★ ★ ★ ★ ★	★ ★
Best for business logic	★ ★ ★ ★ ★	★
Best for numeric kernels	★ ★ ★ ★	★ ★ ★ ★ ★

17. Most Important Alternative — JIT Auto-Vectorization

For many workloads, the first alternative to the Vector API is simply:

```
for (int i = 0; i < array.length; i++) {
    result[i] = array[i] * 2;
}
```

Modern JVMs can recognize certain loops and automatically generate vector instructions.

Therefore:

Simple Java loop
↓
JIT compiler

↓
Auto-vectorization
↓
SIMD instructions

This is extremely important.

Do not assume you need Vector API merely because SIMD is required.

First write clear Java and benchmark it.

Use Vector API when explicit vector operations provide something the JIT isn't reliably giving you.

18. Alternate Features

Requirement	Vector API	Alternative	When Alternative Is Better
Simple numeric loop	Vector API	Ordinary Java loop	Let JIT optimize it
SIMD-friendly loop	Vector API	JIT auto-vectorization	When JVM already generates efficient SIMD
Multi-core parallelism	Vector API	Parallel Streams	When work is coarse-grained and independent
CPU thread parallelism	Vector API	ExecutorService / virtual threads	For independent tasks/I/O
Matrix operations	Vector API	BLAS/native library	Mature high-performance numerical libraries
ML computation	Vector API	GPU/ML frameworks	Large tensor workloads
Image processing	Vector API	Image-processing libraries	When algorithms are already optimized
Native SIMD intrinsics	Vector API	JNI	When existing native library must be reused
Native C/C++ numerical code	Vector API	FFM API	When native library is required
Simple array	Vector API	Streams/loops	When readability

transformation			matters more
Distributed numerical processing	Vector API	Distributed compute framework	When data exceeds one machine

19. Vector API vs JNI

Historically, developers might write:

```

Java
↓
JNI
↓
C/C++
↓
CPU SIMD instructions

```

With Vector API:

```

Java
↓
Vector API
↓
JIT
↓
CPU SIMD instructions

```

Aspect	Vector API	JNI
Native code required	✗	✓
SIMD operations	✓	✓
Java type safety	Higher	Lower
Portability	Higher	Lower
Maintenance	Easier	Harder
Native library reuse	✗	✓
JVM optimization	Strong opportunity	Limited across boundary
Best for	New Java SIMD algorithms	Existing native libraries

--	--	--

20. Vector API vs FFM API

Aspect	Vector API	FFM API
Primary purpose	SIMD computation	Native function/memory access
CPU vector operations	✓	Not its primary purpose
Call C/C++ functions	✗	✓
Native memory	✗	✓
Java SIMD implementation	✓	✗
Native library integration	✗	✓
Best use	Numeric computation inside Java	Interoperability with native code

So:

Need SIMD inside Java?



Vector API

Need to call native C/C++?



FFM API

21. Performance Considerations

Factor	Impact
Large arrays	☐ Usually favorable
Independent operations	☐ Excellent
Primitive numeric data	☐ Excellent
Contiguous memory	☐ Excellent

CPU with strong SIMD support	☐ Excellent
Small arrays	☐ Often little benefit
Branch-heavy algorithm	☐ Can reduce benefit
Random memory access	☐ Can reduce benefit
Data dependencies	☐ Poor SIMD candidate
Object-heavy data	☐ Poor fit
I/O-bound workload	☐ No meaningful benefit
JIT auto-vectorization already effective	☐ Manual API may add little
Poorly chosen vector width	☐ Can reduce portability/performance

22. Data Layout Matters

Vectorization works best with data such as:

`float[]` values;

rather than:

`List<Measurement>`

where:

```
class Measurement {
    float value;
    String name;
    ...
}
```

Conceptually:

Good SIMD layout:

```
[ v1 v2 v3 v4 v5 v6 v7 v8 ]
```

Poor SIMD layout:

```
[Object] [Object] [Object] [Object] ...
```

This is one reason high-performance numerical systems often use **data-oriented layouts**.

23. Vector API and Cache Efficiency

Vectorization isn't only about CPU instructions.

Suppose:

Memory
↓
Cache
↓
Vector registers
↓
SIMD arithmetic

You want data that can be loaded efficiently.

A vectorized algorithm with poor memory locality may still perform poorly.

Therefore:

Vectorization
+
Cache locality
+
Efficient memory access
↓
High performance

24. When Vector API Is a Strong Candidate

Characteristic	Suitability
Millions of values	★ ★ ★ ★ ★
Primitive arrays	★ ★ ★ ★ ★
Same operation per element	★ ★ ★ ★ ★
Independent iterations	★ ★ ★ ★ ★
Few branches	★ ★ ★ ★ ★
Contiguous memory	★ ★ ★ ★ ★

Numeric computation	★ ★ ★ ★ ★
CPU-bound	★ ★ ★ ★ ★
Object-heavy	★
I/O-bound	★
Small input	★
Highly irregular algorithm	★

25. Common Mistakes

Mistake	Why It Is Bad
Vectorizing every loop	Creates unnecessary complexity
Assuming Vector API is always faster	Performance depends on workload
Ignoring JIT auto-vectorization	Simple loops may already be optimized
Benchmarking tiny datasets	Doesn't represent real workload
Ignoring memory bandwidth	CPU SIMD may not be bottleneck
Using it for object-heavy code	Poor fit
Ignoring CPU architecture	Different CPUs have different capabilities
Assuming same speed everywhere	Generated instructions differ
Forgetting tail elements	Array length may not equal vector width
Using preview/incubator APIs casually	API can change
Optimizing before profiling	May optimize the wrong code

26. Recommended Development Process

Step	Action
-------------	---------------

1	Write a simple, correct Java implementation
2	Profile the application
3	Identify CPU-intensive hotspots
4	Determine whether the algorithm is SIMD-friendly
5	Benchmark the ordinary loop
6	Check whether JIT auto-vectorization is sufficient
7	Implement Vector API version
8	Benchmark using realistic datasets
9	Test on target CPU architectures
10	Keep Vector API only if the measurable benefit justifies complexity

27. Alternate Feature Decision Table

If your requirement is...	Prefer
Simple numeric processing	Ordinary Java loop
JIT-friendly loop	Ordinary Java + auto-vectorization
Explicit SIMD operations	Vector API
Multi-core parallel computation	Parallel Stream / ExecutorService
Massive ML tensors	GPU/ML framework
Native C/C++ library	FFM API
Existing native SIMD implementation	FFM API/native library
Matrix algebra	Optimized numerical library
Image manipulation	Specialized image-processing library
High-performance Java numeric kernel	Vector API

28. Vector API vs Alternatives — Overall

Feature	SIMD	Multi-core	Native code	Ease of use	Best Use
Vector API	★ ★ ★ ★ ★	★ ★	✗	★ ★ ★	Java SIMD
Ordinary loop	★ ★ ★ ★ *	✗	✗	★ ★ ★ ★ ★	General computation
Parallel Stream	★ ★	★ ★ ★ ★ ★	✗	★ ★ ★ ★ ★	Coarse parallelism
ExecutorService	★	★ ★ ★ ★ ★	✗	★ ★ ★ ★	Task parallelism
FFM API	★ ★	★ ★	✓	★ ★ ★	Native integration
JNI	★ ★ ★ ★	★ ★ ★	✓	★ ★	Legacy/native libraries
GPU framework	★ ★ ★ ★ ★	★ ★ ★ ★ ★	Usually	★ ★	Massive numerical workloads
Specialized numerical library	★ ★ ★ ★ ★	★ ★ ★ ★ ★	Often	★ ★ ★ ★	Production numerical computing

* An ordinary loop may be automatically vectorized by the JIT.

29. Production Suitability

Application	Vector API Suitability
Spring Boot CRUD	☐ Very low
REST API	☐ Low
Banking transaction processing	☐ Depends on hot numeric calculations
High-frequency analytics	☐ High
Image processing	☐ High
Video processing	☐ High

Audio processing	☐ High
Scientific computing	☐ Excellent
ML inference kernels	☐ High
Signal processing	☐ Excellent
ETL numerical transformation	☐ High
Compression	☐ Potentially high
Cryptography	☐ Algorithm-dependent
Database engine internals	☐ Potentially high
IoT numeric processing	☐ Workload-dependent
Business workflow	☐ Very low

30. Final Recommendation

Scenario	Recommendation
Normal application code	✗ Don't use Vector API
Simple numeric loop	☐ Start with normal Java
JVM already auto-vectorizes loop	☐ Keep normal Java
Large CPU-bound numerical kernel	☐ Consider Vector API
Image/audio/signal processing	☐ Strong candidate
ML/numerical computation	☐ Strong candidate
Large primitive arrays	☐ Strong candidate
Small data	☐ Avoid
Object-heavy workload	☐ Avoid
I/O-bound application	☐ Avoid
Multi-core problem rather than SIMD problem	☐ Use parallelism instead

Existing native library	☐ Consider FFM API
GPU-scale computation	☐ Consider GPU/ML framework
Java 24 project accepting incubator APIs	☐ Benchmark before adopting

Bottom line

Vector API (Ninth Incubator) is a specialized performance tool, not a general-purpose Java feature.

The ideal progression is:

```

Simple Java loop
    ↓
Profile
    ↓
Is it CPU-bound?
    ↓
Is the algorithm SIMD-friendly?
    ↓
Can the JIT auto-vectorize it?
    ↓
    Yes ─────────→ Keep simple Java
    |
    No
    ↓
Consider Vector API
    ↓
Benchmark
    ↓
Keep only if the performance gain justifies complexity

```

Its strongest real-world use cases are **large primitive-array computations, image/audio processing, signal processing, scientific computing, ML kernels, compression, and other CPU-intensive numerical workloads**. For ordinary enterprise application code, **normal Java loops plus JIT optimization are usually the better choice**.

One additional point for your Java-version tracking: **JEP 469 was still an incubator API in Java 24**, so production adoption should account for the possibility of API changes in subsequent JDK releases.

Java 24 — Remove Non-Generational ZGC

Property	Details
Java Version	24
Year	2025
Feature	Remove Non-Generational ZGC
JEP	JEP 490
UniqueClassName	RemoveNonGenerationalZGC
Status	Final / implemented
Type	JVM / Garbage Collector change
Affected GC	ZGC
Change	Non-generational ZGC was removed
Java 24 behavior	ZGC is always generational
Old option	-XX:+ZGenerational
Java 24 status of option	Removed/obsolete; no longer needed
Main goal	Simplify ZGC and focus future development on generational collection

In Java 23, Generational ZGC became the default and non-generational ZGC was deprecated. In Java 24, the non-generational implementation was removed entirely. ([Oracle Docs](#))

1. What Does "Remove Non-Generational ZGC" Mean?

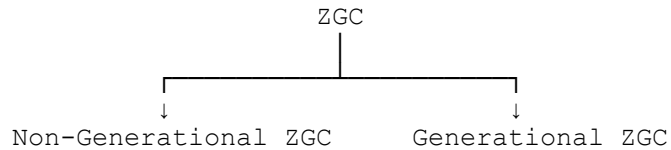
This is important:

Java 24 did not remove ZGC.

It removed **one mode of ZGC**.

Before Java 24

Conceptually:



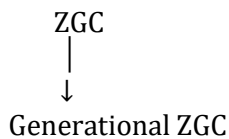
Applications could use:

`-XX:+UseZGC`

and, in releases that supported both modes, select the generational mode with:

`-XX:+ZGenerational`

Java 24



There is no longer a choice between the two modes. Oracle's Java 24 documentation explicitly states that ZGC is generational and that the ZGenerational option has been removed. ([Oracle Docs](#))

2. Why Was Non-Generational ZGC Removed?

Modern Java applications tend to allocate huge numbers of **short-lived objects**.

For example:

HTTP Request
↓
Temporary objects
↓
JSON objects
↓
DTOs
↓
Response objects
↓
Most become unreachable

Generational GC exploits this behavior.

Instead of treating every object equally:

All objects



Collect everything similarly

generational collection separates objects broadly into:

Young objects



Likely to die quickly

Old objects



Likely to survive

This allows the GC to concentrate more effort on young objects and reduce unnecessary work on objects that are likely to remain alive.

ZGC's generational implementation became the default in Java 23, and JEP 490 removed the older non-generational implementation in Java 24. ([OpenJDK Mail](#))

3. Advantages

Advantage	Explanation
Simpler ZGC implementation	Only one ZGC mode needs to be maintained
Better focus for future development	ZGC development can concentrate on generational collection
Better handling of short-lived objects	Young objects can be collected more frequently
Potentially lower GC overhead	Less work may be spent repeatedly processing long-lived objects
Potentially better allocation throughput	Generational collection is well suited to high allocation rates
Potentially lower memory overhead	Generational collection can reduce unnecessary GC work
No mode-selection complexity	Developers no longer need to decide between generational and non-generational ZGC
Simpler tuning	One ZGC architecture instead of two

Consistent ZGC behavior	Applications using ZGC get generational behavior
Better long-term maintainability	Removes duplicated/legacy implementation paths

The ZGC tuning guide describes modern ZGC as adaptive: it dynamically resizes generations, adjusts the number of GC threads, and adjusts tenuring thresholds. ([Oracle Docs](#))

4. Disadvantages

Disadvantage	Explanation
Loss of the non-generational option	Applications cannot choose the old mode
Legacy configurations may need cleanup	Scripts containing <code>-XX:+ZGenerational</code> should be reviewed
Workloads that preferred non-generational behavior lose that option	There is no fallback within ZGC
Migration testing may be necessary	GC behavior changes when moving older applications to Java 24
More complex generational implementation	Generational GC introduces additional GC concepts
Possible workload-specific differences	Not every workload benefits equally from generational collection
Tuning assumptions may change	Old ZGC tuning guidance may no longer apply
ZGC remains memory-sensitive	Adequate heap headroom is still important
Not a universal performance improvement	Actual throughput/latency depends on workload
Cannot revert to non-generational ZGC	If the workload doesn't suit it, another GC must be selected

5. ZGC Before and After

Aspect	Older ZGC	Java 24 ZGC
Generational mode	Optional	Always enabled
Non-generational mode	Available	Removed
-XX:+ZGenerational	Used to explicitly select generational mode	No longer necessary
ZGC architecture	Two modes	One mode
Young generation	Generational mode only	Always present
Old generation	Generational mode only	Always present
Tuning complexity	Higher	Lower
Future ZGC development	Two modes	Focused on generational ZGC
Main use case	Low-latency applications	Low-latency applications

6. Basic Configuration in Java 24

The normal way to enable ZGC is simply:

```
java -XX:+UseZGC -jar application.jar
```

You **do not** need:

```
-XX:+ZGenerational
```

because Java 24 ZGC is generational by definition. ([Oracle Docs](#))

7. What Happens to Old Startup Scripts?

Older configuration

```
java \
-XX:+UseZGC \
-XX:+ZGenerational \
-Xmx16g \
-jar application.jar
```

Java 24 configuration

```
java \  
-XX:+UseZGC \  
-Xmx16g \  
-jar application.jar
```

The important migration step is to **remove the obsolete generational-mode switch**.

8. When to Use

Because this is a **removal**, you don't really "use" the feature itself. You use **ZGC in Java 24**, which is automatically generational.

Situation	Use Java 24 Generational ZGC?	Reason
Very low latency requirement	✓ Excellent candidate	ZGC is designed for low pauses
Large heap	✓ Excellent candidate	ZGC scales to very large heaps
High object allocation rate	✓ Excellent candidate	Generational GC handles short-lived objects well
Microservices with high request volume	✓ Good	Many temporary request objects
Large in-memory application	✓ Good	Low-latency collection
Real-time-ish analytics	✓ Good	GC pauses are designed to remain low
Large caches	□ Good candidate	Low-pause behavior can be valuable
Trading/financial analytics	□ Potentially	Latency may be important
Large messaging application	□ Potentially	High allocation rates
Long-running server	□ Good	Stable low-pause GC
Small application with modest heap	□ Maybe	G1 may be simpler

Throughput-only batch processing	☐ Benchmark	Parallel GC may be better
Tiny CLI application	☐ Usually unnecessary	ZGC overhead may not be justified

ZGC is designed as a scalable low-latency collector, with pauses intended to remain very short and largely independent of heap size. Oracle documents support for heaps ranging from hundreds of megabytes up to 16 TB. ([Oracle Docs](#))

9. When NOT to Use ZGC

Situation	Avoid ZGC?	Better Choice
Small heap application	☐ Often	G1 / Serial
Simple CLI tool	☐ Often	Serial / G1
Maximum batch throughput is priority	☐ Consider	Parallel GC
Latency isn't important	☐ Consider	G1 / Parallel
Extremely CPU-constrained environment	☐ Benchmark	G1 / Parallel
Application is already excellent with G1	✗ Don't switch automatically	Keep G1
Specialized low-latency workload	✓ Use ZGC	ZGC
Huge heap with strict latency requirements	✗ Don't avoid	ZGC is strong candidate

10. Real-Time Use Case — High-Throughput REST API

Imagine:

10,000 requests/sec

↓

JSON parsing

↓
DTO creation
↓
Business processing
↓
Temporary objects
↓
Response
↓
Most temporary objects become garbage

A generational collector can exploit this:

Young generation
↓
Many short-lived request objects
↓
Frequent collection

Old generation
↓
Long-lived application objects
↓
Collected less frequently

This is a natural workload for generational ZGC.

11. Real-Time Use Case — Microservices

Suppose an order service creates many temporary objects:

HTTP Request
↓
Order DTO
↓
Validation objects
↓
Pricing objects
↓
JSON objects
↓
Response DTO

Most of these objects live for only milliseconds or seconds.

Generational ZGC can distinguish:

Short-lived
↓
Young generation

Long-lived
↓
Old generation

This can reduce the amount of GC work required compared with treating all objects uniformly.

12. Real-Time Use Case — Large In-Memory Application

Suppose:

-Xmx128g

and the application maintains:

Large caches
+
Long-lived objects
+
Short-lived request objects

ZGC is attractive because it is designed for large heaps while maintaining low pause times. ([Oracle Docs](#))

The generational model provides:

128 GB heap
├── Young objects
└── Old objects

rather than treating the heap as one undifferentiated population.

13. Real-Time Use Case — Financial Analytics

Consider a financial analytics engine:

Market data
↓
Create temporary calculations

↓
Calculate indicators
↓
Generate result
↓
Temporary objects become garbage

If the application has:

- large heap,
- high allocation rate,
- strict latency requirements,

Generational ZGC can be a strong candidate.

However, **benchmark against G1 and other collectors** rather than assuming ZGC will always be faster.

14. Real-Time Use Case — Messaging System

Imagine a messaging service:

Message
↓
Deserialize
↓
Validate
↓
Transform
↓
Route
↓
Serialize

Each message can generate many short-lived objects.

For millions of messages:

Message 1 → temporary objects → garbage
Message 2 → temporary objects → garbage
Message 3 → temporary objects → garbage
...

Generational GC is particularly well suited to this kind of allocation pattern.

15. Real-Time Use Case — Large Java Cache

Consider:

Application

- Long-lived cache entries
- Request objects
- Temporary lookup objects
- Serialization objects

The cache entries may survive for hours, while request objects may survive for milliseconds.

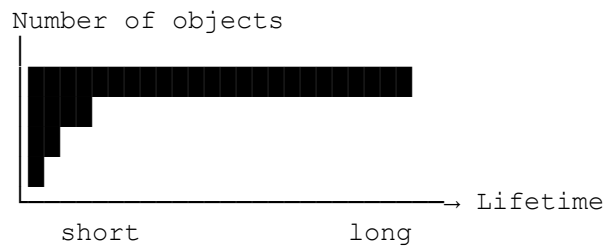
Generational ZGC can separate those lifetime patterns.

16. Generational ZGC vs Non-Generational ZGC

Characteristic	Non-Generational ZGC	Generational ZGC
Young generation	✗	✓
Old generation	✗	✓
Exploits object lifetime	Less directly	✓
Short-lived object optimization	Limited	✓
Long-lived object separation	✗	✓
Java 23	Available but deprecated	Default
Java 24	✗ Removed	Only mode
Future ZGC development	✗ Legacy	✓ Primary
Recommendation in Java 24	✗ Impossible	✓ Use

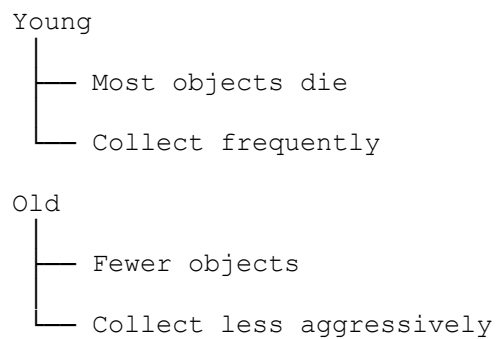
17. Why Generational GC Helps

A typical Java application has an object lifetime distribution similar to:



A large percentage of objects may become unreachable relatively quickly.

Generational collection exploits this observation:



This is one of the fundamental reasons generational collectors can reduce GC overhead for allocation-heavy applications.

18. ZGC vs G1

Aspect	Generational ZGC	G1
Low latency	★★★★★	★★★★
Very large heaps	★★★★★	★★★★
General-purpose	★★★★	★★★★★
Throughput	★★★★	★★★★
Pause-time target	Very low	Low
Complexity	Moderate	Moderate

Default GC	✗	G1 remains default
Good for normal enterprise apps	★★★★	★★★★
Good for latency-sensitive apps	★★★★★	★★★★
Small heap	★★★	★★★★★
Large allocation rate	★★★★★	★★★★

Oracle describes G1 as the default collector and ZGC as a low-latency collector designed for applications with stringent pause-time requirements. ([Oracle Docs](#))

19. ZGC vs Shenandoah

Aspect	Generational ZGC	Generational Shenandoah
Low latency	★★★★★	★★★★★
Large heaps	★★★★★	★★★★★
Generational support	✓	Depends on JDK/version
Main implementation focus	Generational ZGC	Generational Shenandoah
Default in Java 24	✗	✗
Good alternative	—	✓
Best choice	Benchmark	Benchmark

For Java 24 specifically, ZGC's non-generational mode is gone, whereas Shenandoah had its own generational development path. Do not treat the two collectors as identical; benchmark the application's actual workload.

20. Alternate Features

Requirement	Generational ZGC	Alternative	When Alternative Is Better
-------------	------------------	-------------	----------------------------

Very low GC pauses	✓	Shenandoah	When its workload characteristics are favorable
General enterprise application	✓	G1	Usually the first GC to evaluate
Maximum throughput	□	Parallel GC	Batch/throughput-focused workloads
Small application	□	G1/Serial	Lower complexity
Huge heap + low latency	★★★★★	Shenandoah	Benchmark against ZGC
High allocation rate	★★★★★	G1/Shenandoah	Depends on allocation pattern
Short-lived object heavy workload	★★★★★	G1	Also generational
Latency-sensitive microservice	★★★★★	G1/Shenandoah	Benchmark p99/p999 latency
Batch processing	□	Parallel GC	Throughput is more important
Small CLI	□	Serial GC	Startup/simplicity
Moderate heap	□	G1	General-purpose choice

21. ZGC vs G1 vs Parallel vs Shenandoah

Requirement	Best Starting Point
General Java application	G1
Very low pause times	ZGC
Very large heap + low latency	ZGC
Alternative low-latency collector	Shenandoah
Maximum batch throughput	Parallel GC

Tiny/simple application	Serial GC
High allocation + strict latency	ZGC
Unknown workload	G1 , then benchmark alternatives

22. Migration from Java 23 to Java 24

Suppose your Java 23 configuration is:

```
java \
-XX:+UseZGC \
-XX:+ZGenerational \
-Xms16g \
-Xmx32g \
-jar application.jar
```

Java 24:

```
java \
-XX:+UseZGC \
-Xms16g \
-Xmx32g \
-jar application.jar
```

Migration table

Migration item	Action
-XX:+UseZGC	✓ Keep
-XX:+ZGenerational	✗ Remove
ZGC tuning	Review
GC logs	Review
Latency benchmarks	Re-run
Throughput benchmarks	Re-run
Heap sizing	Review

Allocation rate	Monitor
GC pause distribution	Monitor
Application correctness	Test

23. Important ZGC Tuning Point

ZGC's most important tuning parameter is generally the maximum heap size:

-Xmx

Oracle's documentation emphasizes that ZGC needs enough heap space for the application's live set **plus headroom for allocations while concurrent GC is running**. Generally, more heap headroom gives ZGC more room to operate, although excessive memory allocation is wasteful. ([Oracle Docs](#))

Example:

```
java \
-XX:+UseZGC \
-Xms16g \
-Xmx32g \
-jar application.jar
```

Don't blindly copy these values—size them from production allocation rate, live set, latency requirements, and available RAM.

24. Performance Considerations

Factor	Expected Impact
High allocation rate	☐ Generational ZGC can be advantageous
Many short-lived objects	☐ Strong candidate
Very large heap	☐ Strong candidate
Strict latency requirements	☐ Strong candidate
Large long-lived object population	☐ Generational separation can help
Small heap	☐ ZGC may be unnecessary
CPU constrained	☐ Benchmark

Low allocation rate	☐ Benefits may be smaller
Batch throughput only	☐ Compare with Parallel/G1
Huge memory headroom	☐ Helps concurrent operation
Poorly sized heap	☐ Can cause allocation pressure

25. What This Feature Does NOT Mean

Misconception	Reality
"ZGC was removed"	✗ No
"Generational GC was removed"	✗ No
"All Java GC is generational now"	✗ No
"G1 was replaced by ZGC"	✗ No
"ZGC automatically gives zero pauses"	✗ No
"ZGC always outperforms G1"	✗ No
"I need -XX:+ZGenerational in Java 24"	✗ No
"Non-generational ZGC can still be selected"	✗ No
"This changes application source code"	Usually ✗
"This is a new API"	✗ It is a GC/JVM change

26. Best-Practice Decision Table

Question	Recommendation
Do you currently use ZGC on Java 24?	✓ It is automatically generational

Do you have -XX:+ZGenerational?	☒ Remove it
Do you need extremely low latency?	☐ Evaluate ZGC
Do you have a huge heap?	☐ Evaluate ZGC
Do you create many short-lived objects?	☐ Strong candidate
Is the application small?	☐ G1 may be simpler
Is throughput more important than latency?	☐ Compare Parallel/G1
Are you already happy with G1?	☐ Don't switch without measurements
Are you migrating from Java 23?	☐ Test ZGC behavior
Do you want non-generational ZGC?	☐ Not available in Java 24

27. Real-Time Scenario Comparison

Application	Recommended GC Starting Point	Why
Spring Boot REST API, moderate heap	G1	General-purpose
Spring Boot API, huge heap + strict latency	ZGC	Low latency
High-throughput messaging	ZGC/G1	Allocation-heavy
Large financial analytics engine	ZGC	Latency + large heap
Batch ETL	Parallel/G1	Throughput may dominate
Large in-memory cache	ZGC	Large heap + latency
Real-time-ish telemetry	ZGC	Low pause requirements
Small command-line utility	Serial/G1	ZGC unnecessary

Large microservice platform	Benchmark ZGC/G1	Workload-dependent
Scientific computing	ZGC/G1/Parallel	Depends on allocation behavior

28. Advantages vs Disadvantages — Executive Summary

Category	Advantage	Disadvantage
Architecture	One ZGC implementation	No non-generational fallback
GC behavior	Better suited to object-lifetime differences	More sophisticated GC behavior
Performance	Potentially lower GC overhead	Not guaranteed for every workload
Latency	Retains ZGC's low-pause objective	Requires sufficient heap headroom
Configuration	Simpler	Old configurations must be updated
Maintenance	Less implementation duplication	Migration may require testing
Future development	Focused on generational ZGC	Legacy workloads lose choice
Cloud applications	Good for allocation-heavy services	Memory sizing remains important

29. Final Recommendation

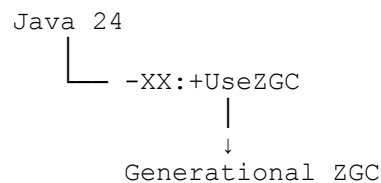
Scenario	Recommendation
Java 24 + ZGC	☐ Use generational ZGC automatically
Existing -XX:+ZGenerational	☐ Remove the flag
Large heap + strict latency	☐ Strong ZGC candidate

High allocation rate	☐ Strong ZGC candidate
Many short-lived objects	☐ Strong ZGC candidate
Small application	☐ Prefer G1/Serial unless measurements justify ZGC
Throughput-oriented batch	☐ Evaluate Parallel/G1
Existing G1 application performing well	☐ Don't migrate without benchmarking
Need non-generational ZGC	☐ Not possible in Java 24
Migration from Java 23	☐ Remove obsolete option and benchmark

Bottom line

JEP 490 is primarily a simplification and consolidation change rather than a new garbage collector. Java 24 removes the old **non-generational ZGC** and makes **generational ZGC the only ZGC mode.** ([Oracle Docs](#))

The practical Java 24 rule is simply:



You no longer choose between generational and non-generational ZGC.

For a **large, allocation-heavy, latency-sensitive application**, generational ZGC is a strong candidate. For a normal enterprise application, **G1 remains an excellent default starting point**, and you should switch to ZGC only when measurements show that its low-latency characteristics are valuable for your workload.

Java 24 — Synchronize Virtual Threads without Pinning

Property	Details
Java Version	24

Year	2025
Feature	Synchronize Virtual Threads without Pinning
JEP	JEP 491
UniqueClassName	VirtualThreadSynchronization
Status	Final
Primary area	Virtual Threads / JVM / Concurrency
Main purpose	Allow virtual threads blocked inside synchronized methods/blocks to unmount from their carrier threads instead of becoming pinned
Main benefit	Makes existing synchronized code much more virtual-thread friendly
Application API changes	Usually none
Related feature	Virtual Threads (JEP 444, Java 21)

Important

Before Java 24, a virtual thread could become **pinned** to its carrier platform thread when it blocked inside a synchronized block or method.

Java 24 removes this major limitation: a virtual thread can now block in most synchronized code and **unmount from its carrier**, allowing the carrier to run another virtual thread.

1. What Is Virtual-Thread Pinning?

A virtual thread normally works like this:

```

Virtual Thread
  ↓
Carrier Platform Thread
  ↓
CPU

```

When the virtual thread performs a blocking operation, Java can generally:

```

Virtual Thread

```

↓
BLOCKED
↓
Unmount from carrier
↓
Carrier executes another virtual thread

This is one of the major advantages of virtual threads.

However, before Java 24:

```
synchronized void process() {  
    blockingOperation();  
}
```

could cause:

Virtual Thread
↓
synchronized
↓
blocking operation
↓
PINNED to carrier
↓
Carrier cannot efficiently run another virtual thread

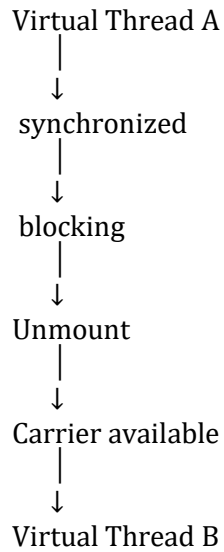
That could reduce scalability when many virtual threads performed blocking operations while holding intrinsic monitors.

2. What Java 24 Changes

Before Java 24

Virtual Thread A
↓
synchronized
↓
blocking
↓
X
↓
Carrier remains occupied

Java 24



This means Java 24 removes the need for developers to replace every synchronized block simply because an application uses virtual threads.

3. Simple Example

Consider:

```
class Inventory {  
    private int stock = 100;  
  
    synchronized void reserve(int quantity) {  
        checkInventory();  
  
        stock -= quantity;  
    }  
  
    private void checkInventory() {  
        // potentially blocking operation  
    }  
}
```

A virtual thread calling:

```
Thread.startVirtualThread(  
    () -> inventory.reserve(10)  
);
```

could previously encounter pinning if it blocked while holding the intrinsic monitor.

In Java 24, the JVM can generally unmount the virtual thread while it is blocked.

4. Advantages

Advantage	Explanation
Eliminates a major virtual-thread limitation	synchronized no longer inherently causes pinning during blocking
Better virtual-thread scalability	Carrier threads can be freed while virtual threads wait
Existing synchronized code becomes more suitable	Less need for large-scale synchronization rewrites
No source-code migration in most cases	Existing synchronized methods/blocks continue to work
Improves blocking workloads	Particularly useful when synchronized code performs blocking operations
Simplifies virtual-thread adoption	Developers don't have to automatically replace synchronized with ReentrantLock
Preserves familiar Java synchronization	Intrinsic monitors remain available
Better carrier utilization	Carrier threads can execute other virtual threads
Reduces unnecessary lock migration	Some applications can keep existing monitor-based synchronization
Works transparently	JVM handles the unmounting behavior
Improves compatibility with legacy libraries	Existing synchronized code becomes less problematic
Makes virtual threads more broadly applicable	More traditional Java code can run efficiently on virtual threads

JEP 491 specifically aims to make virtual threads cooperate with synchronized blocks and methods in the same way they already cooperate with `java.util.concurrent` locking mechanisms.

5. Disadvantages

Disadvantage	Explanation
Doesn't make synchronization free	Lock contention still costs performance
Blocking while holding a lock is still bad design	Unpinning doesn't eliminate lock contention
Some rare pinning scenarios remain	Native/foreign-function interactions and certain JVM situations still require attention
Contention can still become a bottleneck	Many virtual threads competing for one monitor can serialize work
Doesn't solve deadlocks	Incorrect synchronization can still deadlock
Doesn't eliminate race conditions	Synchronization is still necessary for shared mutable state
Doesn't make CPU-intensive code scalable	Virtual threads are primarily beneficial for high-concurrency blocking workloads
Lock scope still matters	Holding a monitor across slow operations can reduce throughput
Developers can become overconfident	"No pinning" does not mean "no synchronization problem"
Performance still depends on workload	Benchmarking remains necessary

6. The Most Important Difference

Before Java 24	Java 24
synchronized + blocking could pin virtual thread	synchronized + blocking generally can unmount
Developers often considered replacing monitors	Less need to replace them solely to avoid pinning
Carrier could remain occupied	Carrier can become available
Virtual-thread scalability could suffer	Better scalability
ReentrantLock sometimes used as workaround	synchronized is generally viable again

7. When to Use

The feature itself is transparent—you don't explicitly enable it.

The practical question is:

When should I use `synchronized` with virtual threads?

Situation	Use <code>synchronized</code> ?	Reason
Protecting shared mutable state	✓ Yes	Still a valid synchronization mechanism
Short critical section	✓ Excellent	Simple and efficient
Existing synchronized application	✓ Yes	Java 24 handles blocking more effectively
Legacy synchronized library	✓ Good	Less concern about virtual-thread pinning
Virtual-thread web server	✓ Yes	<code>synchronized</code> is no longer inherently problematic
Simple cache protection	✓ Good	Straightforward synchronization
Thread-safe counters/state	✓ Good	When appropriate
Blocking I/O inside synchronized region	□ Possible	Pinning is addressed, but lock contention remains
Long-running critical section	□ Carefully	Other threads may wait for the monitor
Highly contended lock	□ Carefully	Can become throughput bottleneck
Complex lock coordination	□ Consider Lock APIs	Explicit locking may be more appropriate

8. When NOT to Use

Situation	Avoid <code>synchronized</code> ?	Better Choice
-----------	-----------------------------------	---------------

Extremely high lock contention	✓ Consider alternatives	ReentrantLock, concurrent collections, atomics
Long critical section	✓ Usually	Redesign synchronization
Lock held during slow external operation	☐ Avoid if possible	Move blocking operation outside lock
Need timed lock acquisition	✓	ReentrantLock.tryLock()
Need interruptible lock acquisition	✓	ReentrantLock.lockInterruptibly()
Need multiple conditions	☐	Condition
Lock-free counter	✗ Don't synchronize unnecessarily	AtomicLong, LongAdder
Read-heavy shared data	☐	ReentrantReadWriteLock / immutable design
No shared mutable state	✗	No synchronization required
CPU-heavy parallel computation	✗	Appropriate executor/parallelism

9. Real-Time Use Case — Web Application

Imagine a web application handling:

100,000 concurrent requests

with virtual threads:

```
Request 1 → Virtual Thread
Request 2 → Virtual Thread
Request 3 → Virtual Thread
...
Request N → Virtual Thread
```

Suppose a shared cache is protected:

```
class Cache {
```

```

private final Map<String, String> data = new HashMap<>();

synchronized String get(String key) {

    return loadFromCacheOrBackend(key);
}
}

```

If the operation blocks, Java 24 can unmount the virtual thread rather than keeping the carrier pinned merely because the code is inside synchronized.

This makes migrating existing synchronized code to virtual threads considerably easier.

10. Real-Time Use Case — Database Access

Consider:

```

class AccountService {

    synchronized void updateAccount(long accountId) {

        Account account =
            database.load(accountId);

        account.update();

        database.save(account);
    }
}

```

The database operations may block.

Before Java 24

```

Virtual Thread
  ↓
synchronized
  ↓
DB call
  ↓
Potential pinning
  ↓
Carrier occupied

```

Java 24

Virtual Thread
↓
synchronized
↓
DB call
↓
Virtual thread can unmount
↓
Carrier available

However, there's an important design problem:

The lock is still held during the database operation.

So other threads needing the same monitor can still be blocked.

Therefore, Java 24 fixes the **carrier-thread pinning problem**, not the **lock-contention problem**.

11. Real-Time Use Case — Legacy Enterprise Application

Suppose an old enterprise system contains:

```
synchronized void processOrder(Order order) {  
    validate(order);  
    callPaymentService();  
    updateInventory();  
    saveOrder();  
}
```

The application has thousands of such synchronized methods.

Previously, migrating to virtual threads could require substantial investigation and potential conversion of synchronization mechanisms.

Java 24 makes this much easier:

Legacy synchronized code
↓
Java 24
↓
Virtual-thread compatible

This is one of the most practically valuable aspects of JEP 491.

12. Real-Time Use Case — Shared Cache

Example:

```
class ProductCache {  
  
    private final Map<Long, Product> cache =  
        new HashMap<>();  
  
    synchronized Product get(long id) {  
  
        Product product = cache.get(id);  
  
        if (product == null) {  
            product = loadFromDatabase(id);  
            cache.put(id, product);  
        }  
  
        return product;  
    }  
}
```

Java 24 avoids the old intrinsic-monitor pinning problem while the virtual thread is blocked.

But there is still a scalability issue:

```
1000 virtual threads  
    ↓  
    one monitor  
    ↓  
serialized access
```

So the next optimization may be cache design rather than changing the synchronization primitive.

13. Real-Time Use Case — Message Processing

Imagine:

```
class MessageProcessor {  
  
    synchronized void process(Message message) {  
  
        updateState(message);  
  
        sendAcknowledgement();  
    }  
}
```

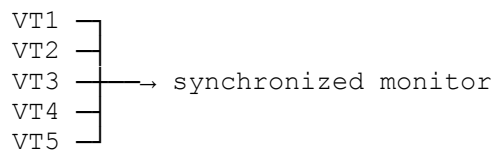
```
}  
}
```

With thousands of virtual-thread-based message handlers:

```
Message 1 → VT1  
Message 2 → VT2  
Message 3 → VT3  
...
```

Java 24 prevents synchronized itself from being the old carrier-pinning limitation.

However, if every message requires the same monitor:



the application is still effectively serializing that portion of the workload.

14. Real-Time Use Case — File Processing

Suppose:

```
synchronized void updateIndex(String file) {  
    readFile(file);  
    updateIndexData();  
}
```

If `readFile()` blocks:

Old concern

```
Virtual Thread  
  ↓  
synchronized  
  ↓  
File I/O  
  ↓  
Carrier pinned
```

Java 24

Virtual Thread
 ↓
 synchronized
 ↓
 File I/O
 ↓
 Unmount
 ↓
 Carrier available

Again, the synchronization design should still be reviewed if many operations compete for the same lock.

15. synchronized vs ReentrantLock

One of the major motivations behind JEP 491 is reducing the need to replace synchronized purely because of virtual-thread pinning.

Aspect	synchronized	ReentrantLock
Syntax	Simple	More verbose
Automatic unlock	✓	✗ Must unlock
try/finally required	✗	✓
Reentrant	✓	✓
Condition variables	wait/notify	Condition
Timed acquisition	✗	✓
Interruptible acquisition	Limited	✓
Java 24 virtual-thread blocking	✓ Unmounting supported	✓
Pinning concern from monitor blocking	Greatly reduced	Not the same issue
Simplicity	★★★★★	★★★
Advanced locking	★★	★★★★★

Before Java 24

Developers sometimes used:

```
ReentrantLock lock = new ReentrantLock();
```

```
lock.lock();
```

```
try {  
    blockingOperation();  
} finally {  
    lock.unlock();  
}
```

specifically to avoid monitor pinning.

Java 24

That workaround is much less necessary merely for avoiding pinning.

16. But ReentrantLock Still Has Advantages

Don't interpret JEP 491 as:

"synchronized is always better than ReentrantLock."

That's incorrect.

Use ReentrantLock when you need:

```
lock.tryLock()
```

or:

```
lock.tryLock(1, TimeUnit.SECONDS)
```

or:

```
lock.lockInterruptibly();
```

or multiple:

```
Condition condition = lock.newCondition();
```

So the choice remains based on **locking semantics**, not simply virtual-thread compatibility.

17. Important Remaining Pinning Cases

JEP 491 specifically addresses pinning caused by blocking in synchronized constructs.

It does **not** mean that virtual threads can never be pinned.

Situation	Pinning concern
Blocking inside synchronized	☐ Major issue addressed
ReentrantLock	☐ Designed to cooperate with virtual threads
Native/JNI execution	☐ Can still pin
Foreign Function & Memory API operations	☐ Depends on operation
CPU-bound code	☐ Not a pinning issue, but consumes carrier
Long-running native code	☐ Can occupy carrier
External process interaction	Depends on API/implementation

Therefore, Java 24's message is:

synchronized



No longer inherently dangerous for virtual threads

not:

All blocking operations



Always safe

18. Advantages vs ReentrantLock

Requirement	synchronized Java 24	ReentrantLock
Basic mutual exclusion	★ ★ ★ ★ ★	★ ★ ★ ★ ★
Simple syntax	★ ★ ★ ★ ★	★ ★ ★
Automatic unlock	★ ★ ★ ★ ★	★ ★

Virtual-thread compatibility	★ ★ ★ ★ ★	★ ★ ★ ★ ★
Timed lock	✗	★ ★ ★ ★ ★
Interruptible lock	★ ★	★ ★ ★ ★ ★
Multiple conditions	★ ★	★ ★ ★ ★ ★
Lock polling	✗	★ ★ ★ ★ ★
Legacy code migration	★ ★ ★ ★ ★	★ ★ ★
Advanced concurrency	★ ★ ★	★ ★ ★ ★ ★

19. Performance Considerations

Factor	Impact
Blocking inside synchronized	☐ Pinning issue substantially improved
Lock contention	☐ Still expensive
Long critical section	☐ Still problematic
Short critical section	☐ Usually good
Many virtual threads	☐ Better carrier utilization
High monitor contention	☐ Can still serialize workload
CPU-bound synchronized work	☐ Still consumes carrier
I/O inside synchronized	☐ Better than before for carrier utilization
Large object allocation	Unrelated
Database bottleneck	Still exists
Network bottleneck	Still exists

20. The Most Important Design Principle

Java 24 changes this:

synchronized + blocking
↓
potential carrier pinning

into:

synchronized + blocking
↓
virtual thread can unmount

But **this remains bad:**

synchronized
↓
slow network call
↓
slow database call
↓
slow external service
↓
large critical section

because:

Many threads
↓
same monitor
↓
waiting

So the recommended design is still:

Acquire lock
↓
Perform minimum protected work
↓
Release lock
↓
Perform slow/blocking operation

when correctness allows it.

21. Alternate Features

Requirement	synchronized	Alternative	When Alternative Is
-------------	--------------	-------------	---------------------

			Better
Simple mutual exclusion	✓	ReentrantLock	Need advanced locking
Timed lock	✗	ReentrantLock.tryLock()	Need timeout
Interruptible lock	✗	ReentrantLock.lockInterruptibly()	Need cancellation
Lock-free counter	✗	AtomicLong / LongAdder	Simple atomic updates
Concurrent map	✗	ConcurrentHashMap	Shared map
Producer/consumer	✗	BlockingQueue	Queue-based coordination
Async task coordination	✗	Structured Concurrency	Related subtasks
High-concurrency I/O	✓ with virtual threads	Virtual Threads	Many blocking tasks
Immutable state	✗	Immutable objects	Eliminate synchronization
Read-heavy state	□	ReentrantReadWriteLock	Different read/write requirements
Lock-free algorithms	✗	Atomic/VarHandle APIs	Extremely specialized workloads
Cross-process coordination	✗	Distributed lock/service	Multiple JVMs

22. synchronized vs Atomic Variables

If you have:

```
synchronized void increment() {
    count++;
}
```

and the operation is simply a counter, you may not need a monitor.

Alternative:

```
private final AtomicLong count = new AtomicLong();
```

```
count.incrementAndGet();
```

For highly concurrent counters:

```
private final LongAdder count =  
    new LongAdder();
```

```
count.increment();
```

Requirement	Better Choice
Complex state mutation	synchronized
Single atomic integer/long	AtomicInteger / AtomicLong
High-contention statistics counter	LongAdder
Concurrent map	ConcurrentHashMap
Advanced lock semantics	ReentrantLock

23. Migration from Java 21–23

If you have:

```
synchronized void process() {  
    blockingCall();  
}
```

you **do not automatically need to rewrite it** for Java 24.

Previous migration advice

synchronized

↓

Potential pinning

↓

Consider ReentrantLock

Java 24

synchronized



Virtual thread can unmount when blocked



Keep it if its locking semantics are appropriate

This significantly simplifies virtual-thread migration.

24. Monitoring Virtual Threads

Although Java 24 addresses monitor pinning, you should still profile concurrency problems.

Look for:

High lock contention
Long critical sections
Carrier starvation
Native blocking
CPU saturation

JFR and JVM diagnostics can help identify synchronization and virtual-thread behavior.

The key point is:

Don't replace synchronization based solely on theoretical pinning concerns; measure the actual application.

25. Best-Practice Decision Table

Question	Recommendation
Does code need mutual exclusion?	✓ Use synchronization
Is synchronized already correct?	✓ Keep it
Is the application moving to virtual threads?	✓ Java 24 makes synchronized safer for this use case
Does synchronized code perform blocking I/O?	☐ Pinning concern is substantially reduced
Is lock contention high?	☐ Redesign/benchmark
Is the critical section large?	☐ Reduce it

Need timed acquisition?	☐ ReentrantLock
Need interruptible acquisition?	☐ ReentrantLock
Need atomic counter?	☐ AtomicLong / LongAdder
Need concurrent map?	☐ ConcurrentHashMap
Need no shared mutable state?	☐ Prefer immutability
Need massive CPU computation?	✗ Virtual threads aren't the solution
Calling native code while blocking?	☐ Investigate pinning separately

26. Overall Comparison

Category	Java 24 synchronized with Virtual Threads
Pinning from monitor blocking	☐ Major improvement
Existing code compatibility	☐ Excellent
Simplicity	☐ Excellent
Basic synchronization	☐ Excellent
Virtual-thread scalability	☐ Much better
Lock contention	☐ Still an issue
Long critical sections	☐ Still an issue
Advanced lock control	☐ Use ReentrantLock
Atomic operations	☐ Use atomic classes
CPU parallelism	☐ Not its purpose
Native blocking	☐ Separate consideration

27. Final Recommendation

Scenario	Recommendation
Existing synchronized code + virtual threads	☐ Keep it unless profiling shows a problem
Short synchronized critical section	☐ Excellent
Blocking I/O inside synchronized	☐ Pinning concern is greatly reduced
Large/high-contention monitor	☐ Investigate redesign
Timed lock requirement	☐ Use ReentrantLock
Interruptible lock requirement	☐ Use ReentrantLock
Simple counter	☐ Use atomic classes
Concurrent map	☐ Use ConcurrentHashMap
CPU-intensive task	☐ Use appropriate platform-thread parallelism
Native/JNI blocking	☐ Investigate separately
Migrating Java 21 application to Java 24	☐ Much less need to replace synchronized solely because of virtual threads

Bottom line

JEP 491 is a major improvement for virtual-thread adoption. Its central change is:

BEFORE JAVA 24

Virtual Thread
↓
synchronized
↓
blocking
↓
PINNED
↓
Carrier unavailable

JAVA 24

Virtual Thread
↓

synchronized
↓
blocking
↓
UNMOUNT
↓
Carrier available
↓
Another Virtual Thread

The practical takeaway is **not** "replace synchronized with ReentrantLock when using virtual threads." Instead:

Use synchronized when intrinsic locking is the right synchronization mechanism. Java 24 removes the major carrier-pinning disadvantage of blocking inside synchronized code.

Use ReentrantLock only when you need capabilities that synchronized doesn't provide—such as **timed acquisition, interruptible locking, or advanced condition/lock management**. And remember that **lock contention, deadlocks, oversized critical sections, and blocking native code remain separate performance/design concerns**.

Java 24 — Flexible Constructor Bodies (Third Preview)

Property	Details
Java Version	24
Year	2025
Feature	Flexible Constructor Bodies (Third Preview)
JEP	JEP 492
UniqueClassName	FlexibleConstructorBodies
Status	Third Preview
First Preview	Java 22
Second Preview	Java 23
Main purpose	Allow statements to appear before an explicit constructor invocation such as <code>super(...)</code> or <code>this(...)</code> , provided they don't access

	the object being constructed
Key improvement	Enables validation, normalization, logging, defensive calculations, and argument preparation before calling another constructor
Important restriction	You still cannot access this before the superclass constructor has completed

JEP 492 is the third preview of this feature. The Java 24 version made a refinement concerning statements before constructor invocation and continued the preview rather than making it final.

1. What Problem Does It Solve?

Traditionally, Java requires an explicit constructor invocation:

```
super(...);
```

or:

```
this(...);
```

to be the **first statement** in a constructor.

For example:

```
class Employee extends Person {
    Employee(String name) {
        super(name.trim());
    }
}
```

You cannot traditionally do:

```
class Employee extends Person {
    Employee(String name) {
        name = name.trim(); // traditionally not allowed here
        super(name);
    }
}
```

Flexible Constructor Bodies allow this type of preparation:

```
class Employee extends Person {  
    Employee(String name) {  
        name = name.trim();  
        super(name);  
    }  
}
```

The important distinction is:

Before super()/this()
↓
Can perform certain computations
↓
Cannot use this/super
↓
super()/this()
↓
Normal constructor body

2. Traditional Constructor Structure

Before this feature:

```
class Child extends Parent {  
    Child(String value) {  
        super(value);  
    }  
}
```

Conceptually:

Constructor starts
↓
super()/this()
↓
Constructor body

The language imposed a very strict ordering.

3. Flexible Constructor Structure

With JEP 492:

```
class Child extends Parent {  
  
    Child(String value) {  
  
        value = normalize(value);  
  
        validate(value);  
  
        super(value);  
  
        initialize();  
    }  
}
```

Conceptually:

```
Constructor starts  
↓  
Pre-constructor statements  
↓  
super()/this()  
↓  
Post-constructor statements
```

This gives constructors two logical phases:

```
Phase 1  
Prepare arguments/state  
↓  
super()/this()  
↓  
Phase 2  
Initialize current object
```

4. Advantages

Advantage	Explanation
Allows preprocessing before super()/this()	Constructor arguments can be calculated more naturally
Improves readability	Validation and normalization can occur near the constructor invocation
Reduces awkward static helper methods	Some calculations can be written directly in the

	constructor
Useful for defensive validation	Validate arguments before passing them to the superclass
Supports normalization	Trim, canonicalize, sanitize, or transform values before delegation
Useful with inheritance	Child constructors can prepare superclass arguments
Supports local variables before constructor invocation	Intermediate calculations become easier
Can simplify complex constructor delegation	Especially when <code>super(...)</code> needs multiple computed arguments
Makes constructor logic more expressive	Removes the artificial "must be first statement" restriction
Works well with records/classes using inheritance patterns	Can simplify initialization logic where applicable
Preserves constructor safety rules	this cannot be used before object construction reaches the required point
Can reduce code duplication	Argument preparation can happen once before delegation

5. Disadvantages

Disadvantage	Explanation
Still a preview feature	Not a permanent Java 24 language feature
New constructor rules	Developers must understand what is legal before <code>super()/this()</code>
Potentially confusing	Developers accustomed to <code>super()</code> always being first may initially struggle
Cannot access this freely	The object is not yet fully constructed
Cannot access instance members normally	Instance state isn't available in the normal way
Can encourage overly complex	Too much preprocessing can make constructors

constructors	difficult to understand
May hide constructor dependencies	Complex pre-invocation calculations can obscure the actual superclass call
Preview compilation required	Production builds need preview flags in Java 24
Future syntax/semantics may change	Preview APIs/language features aren't guaranteed to remain identical
Not needed for simple constructors	Traditional constructor syntax is clearer in many cases
Potential initialization-order confusion	More code can execute before the superclass constructor

6. Key Rule — What Can Be Used Before `super()`?

This is the most important part.

Before the explicit constructor invocation, code cannot access the object under construction.

Allowed conceptually

```
class Child extends Parent {
    Child(String name) {
        String normalized = name.trim();
        super(normalized);
    }
}
```

Here:

```
name
↓
trim()
↓
normalized
↓
super(normalized)
```

The calculation doesn't require the partially constructed Child object.

Not allowed

```
class Child extends Parent {  
    private int value = 10;  
  
    Child() {  
        System.out.println(value);  
        super();  
    }  
}
```

Why?

Because:

this.value
↑
requires current object
↑
object not yet initialized through superclass constructor

The fundamental safety rule remains.

7. When to Use

Situation	Use?	Reason
Need to normalize constructor arguments	✓ Excellent	Prepare values before super()
Need to validate arguments	✓ Good	Validate before delegation
Complex superclass arguments	✓ Excellent	Calculate them locally
Need temporary variables for super()	✓ Excellent	Makes argument preparation readable
Parsing constructor input	✓ Good	Parse before superclass invocation

Defensive copying of arguments	✓ Good	Prepare safe values
Canonicalizing strings	✓ Good	Normalize before delegation
Converting units	✓ Good	Calculate required superclass value
Constructor delegation requiring preprocessing	✓ Excellent	Cleaner than external helpers
Simple constructor	✗ Usually unnecessary	Traditional form is clearer
Logic requiring this	✗ No	Object isn't ready
Large business workflow	✗ No	Constructor should remain focused
Expensive I/O	✗ Usually no	Constructors shouldn't perform slow operations
Complex mutable initialization	□ Carefully	Can hurt readability

8. When NOT to Use

Situation	Why Avoid It?	Better Choice
super() takes no arguments	No benefit	Traditional constructor
Simple argument forwarding	Unnecessary	super(value)
Need instance fields before super()	Not allowed	Redesign initialization
Complex business logic	Constructor becomes difficult to maintain	Factory/service method
Database calls	Constructors shouldn't normally perform I/O	Factory/service
Network calls	Same reason	Service/factory
Large validation workflow	Too much constructor responsibility	Validator/factory

Many preprocessing steps	Reduces readability	Static factory method
Public library targeting stable Java	Preview dependency	Conventional constructors
Team isn't using preview features	Avoid unnecessary complexity	Traditional syntax

9. Real-Time Use Case — Input Normalization

Suppose:

```
class Person {
    Person(String name) {
        // superclass initialization
    }
}
```

Child class:

```
class Employee extends Person {
    Employee(String name) {
        String normalized = name.trim().toUpperCase();
        super(normalized);
    }
}
```

Conceptually:

```
" anurag "
↓
trim()
↓
"anurag"
↓
toUpperCase()
↓
"ANURAG"
↓
super("ANURAG")
```

This is a natural use case because the transformation is required **before** superclass initialization.

10. Real-Time Use Case — Validation

Suppose the parent class requires a valid positive amount.

```
class Account {  
  
    Account(long balance) {  
        if (balance < 0) {  
            throw new IllegalArgumentException();  
        }  
    }  
}
```

Child:

```
class SavingsAccount extends Account {  
  
    SavingsAccount(long balance) {  
  
        long normalized = Math.max(0, balance);  
  
        super(normalized);  
    }  
}
```

Or with validation:

```
class SavingsAccount extends Account {  
  
    SavingsAccount(long balance) {  
  
        if (balance < 0) {  
            throw new IllegalArgumentException(  
                "Balance cannot be negative"  
            );  
        }  
  
        super(balance);  
    }  
}
```

The important point is that the child can perform validation before invoking the superclass constructor.

11. Real-Time Use Case — Unit Conversion

Suppose the superclass expects meters:

```
class Distance {  
    Distance(double meters) {  
        // ...  
    }  
}
```

Child constructor receives kilometers:

```
class RoadDistance extends Distance {  
    RoadDistance(double kilometers) {  
        double meters = kilometers * 1000;  
        super(meters);  
    }  
}
```

The constructor becomes:

```
kilometers  
  ↓  
× 1000  
  ↓  
meters  
  ↓  
super(meters)
```

This is a good example of **preparing constructor arguments**.

12. Real-Time Use Case — Multiple Calculations

Suppose:

```
class Rectangle {  
    Rectangle(double width, double height) {  
        // ...  
    }  
}
```

A subclass receives dimensions and a scale:

```

class ScaledRectangle extends Rectangle {

    ScaledRectangle(
        double width,
        double height,
        double scale
    ) {

        double scaledWidth = width * scale;
        double scaledHeight = height * scale;

        super(scaledWidth, scaledHeight);
    }
}

```

This is considerably clearer than trying to put everything directly inside:

```

super(width * scale, height * scale);

```

13. Real-Time Use Case — Defensive Copy / Snapshot

Suppose a superclass accepts a configuration snapshot:

```

class Service {

    Service(Map<String, String> config) {
        // ...
    }
}

```

Child:

```

class PaymentService extends Service {

    PaymentService(Map<String, String> config) {

        Map<String, String> snapshot =
            Map.copyOf(config);

        super(snapshot);
    }
}

```

The child prepares an immutable snapshot before superclass construction.

This can be useful for:

- Configuration
 - Security settings
 - Immutable request metadata
 - Service initialization
-

14. Real-Time Use Case — Parsing

Suppose a superclass accepts a parsed value:

```
class User {  
    User(long userId) {  
        // ...  
    }  
}
```

Child receives a String:

```
class AdminUser extends User {  
    AdminUser(String userId) {  
        long id = Long.parseLong(userId);  
        super(id);  
    }  
}
```

Conceptually:

```
String  
↓  
parse  
↓  
long  
↓  
super(long)
```

This is another strong use case.

15. Constructor Delegation with this()

Flexible constructor bodies can also be useful when delegating to another constructor.

Conceptually:

```

class Employee {

    Employee(String name) {
        this(normalize(name));
    }

    Employee(String name, boolean active) {
        ...
    }
}

```

However, remember that constructor invocation rules still apply.

The main value is that calculations can be performed before constructor invocation when the language rules permit them.

16. Traditional Approach vs Flexible Constructor

Traditional approach

Developers often move preprocessing into a helper:

```

class Employee extends Person {

    Employee(String name) {
        super(normalize(name));
    }

    private static String normalize(String name) {
        return name.trim().toUpperCase();
    }
}

```

Flexible constructor

Conceptually:

```

class Employee extends Person {

    Employee(String name) {

        String normalized =
            name.trim().toUpperCase();

        super(normalized);
    }
}

```

Aspect	Helper Method	Flexible Constructor
Extra method	Required	Not required
Local calculations	Less direct	Direct
Readability	Good	Often better for short calculations
Reuse	Excellent	Limited
Complex transformation	Better	Can become messy
Constructor-focused logic	Good	Excellent

17. Don't Put Too Much Logic Before super()

This is technically possible:

```
class Employee extends Person {
    Employee(String name) {
        validateName(name);

        String normalized = normalize(name);

        AuditRecord audit = createAuditRecord(name);

        saveAuditRecord(audit);

        String result = callExternalService(name);

        super(result);
    }
}
```

But this is **bad constructor design**.

Problems:

```
Constructor
↓
Validation
↓
Database
```

↓
Network
↓
Audit
↓
Superclass

Instead:

Factory/service
↓
prepare data
↓
constructor
↓
super()

Flexible Constructor Bodies are not intended to turn constructors into workflows.

18. Advantages in Inheritance

Problem	Traditional Java	Flexible Constructor
Calculate superclass argument	Awkward/helper method	Local calculation
Validate before superclass	Difficult	Direct
Normalize input	Helper/static method	Direct
Multiple temporary values	Awkward	Natural
Convert units	Inline/helper	Local variable
Defensive copy	Helper	Direct
Parse input	Helper	Direct

19. Disadvantages in Inheritance

Problem	Why It Matters
---------	----------------

Initialization order	Developers must understand what exists before super()
Instance access	Still prohibited before superclass construction
Complex code	Can make constructors hard to read
Side effects	Can make object creation unpredictable
Exception handling	Exceptions can occur before superclass construction
Preview dependency	Not suitable for all production environments
Learning curve	New rules around constructor invocation
Debugging	More executable code exists before superclass construction

20. Alternate Features

There is no single exact replacement for Flexible Constructor Bodies.

Requirement	Flexible Constructor Bodies	Alternative	When Alternative Is Better
Calculate super() arguments	✓	Static helper method	When logic is reusable
Normalize input	✓	Static factory method	When normalization is substantial
Validate object creation	✓	Factory method	Complex validation
Create complex object	?	Builder	Many optional parameters
Parse input	✓	Static factory	Parsing is substantial
Perform business workflow	✗	Service/factory	Better separation of concerns
Convert units	✓	Static helper	Reusable conversion
Defensive copy	✓	Constructor/helper	Simple transformation
Complex initialization	?	Factory	Constructor remains small

Multiple construction strategies	?	Static factories	Clear named creation methods
Simple constructor forwarding	✗	Normal super()	Simpler

21. Flexible Constructor Bodies vs Static Factory

Aspect	Flexible Constructor	Static Factory
Syntax	Constructor	Static method
Can preprocess before super()	✓	N/A
Can return subtype	✗	✓
Can cache instances	✗	✓
Can choose implementation	✗	✓
Constructor inheritance	Works with inheritance	Can avoid complex inheritance
Complex creation	☆☆	☆☆☆☆
Simple preprocessing	☆☆☆☆	☆☆☆
Naming	Constructor has class name	Factory can have descriptive name

For example:

```
Employee.fromExternalId("EMP-100");
```

can communicate intent better than a constructor if creation involves substantial logic.

22. Flexible Constructor Bodies vs Builder

Requirement	Flexible Constructor	Builder
-------------	----------------------	---------

2-3 simple values	★★★★★	★★
Many optional fields	★★	★★★★★
Validation	★★★★	★★★★★
Complex construction	★★★	★★★★★
Inheritance preprocessing	★★★★★	★★★
Readability with many parameters	★★	★★★★★
Immutable object construction	★★★★	★★★★★

23. Flexible Constructor Bodies vs Factory Method

Requirement	Constructor	Factory
Simple object creation	★★★★★	★★★★
Complex preparation	★★★	★★★★★
Multiple implementations	★★	★★★★★
Caching	✗	✓
Naming creation operation	★★	★★★★★
Inheritance super() preparation	★★★★★	★★★
Object creation control	★★★	★★★★★

24. Performance Considerations

Aspect	Impact
Local arithmetic before	Usually negligible

super()	
String normalization	Depends on allocation/work
Defensive copying	Can allocate memory
Parsing	CPU cost depends on input
Complex calculations	May increase construction time
Network/database operations	☒ Should generally be avoided
Helper method vs inline code	Usually not a reason to choose one; JIT may inline small helpers
Preview syntax	No inherent performance guarantee

Important: Flexible Constructor Bodies are primarily a **language expressiveness feature**, not a performance optimization.

25. Best Practices

Rule	Recommendation
Keep preprocessing short	✓
Calculate superclass arguments locally	✓
Validate external arguments	✓
Normalize values	✓
Use temporary variables for clarity	✓
Avoid I/O	✓
Avoid database calls	✓
Avoid network calls	✓
Avoid complex business logic	✓
Don't access this before super()	☒ Mandatory
Don't use it just because it is available	✓

Prefer factory methods for complex construction	✓

26. Decision Table

Question	Recommendation
Do I need to calculate a super() argument?	☑ Use it
Do I need to validate input before super()?	☑ Use it
Do I need normalization before super()?	☑ Use it
Do I need several local variables to prepare arguments?	☑ Good candidate
Is the constructor only super(x)?	☑ Keep traditional form
Does the logic require this?	☑ Cannot do it before super()
Does the logic perform I/O?	☑ Move it outside
Is construction complex?	☑ Consider factory/builder
Is the logic reusable?	☑ Static helper may be better
Is Java 24 preview acceptable?	☑ Required
Is long-term stable Java required?	☑ Avoid preview dependency

27. Feature Comparison

Category	Flexible Constructor Bodies	Static Factory	Builder	Helper Method
Preprocess before super()	★★★★★	★★	★★	★★★★
Simple construction	★★★★★	★★★★	★★	★★★

Complex construction	☆☆☆	☆☆☆☆☆	☆☆☆☆☆	☆☆☆
Many optional fields	☆☆	☆☆☆	☆☆☆☆☆	☆☆
Reusability	☆☆	☆☆☆☆☆	☆☆☆☆	☆☆☆☆☆
Readability	☆☆☆☆	☆☆☆☆☆	☆☆☆☆☆	☆☆☆☆
Object creation control	☆☆☆	☆☆☆☆☆	☆☆☆☆☆	☆☆
Inheritance integration	☆☆☆☆☆	☆☆☆	☆☆☆	☆☆☆☆
Preview dependency	?	×	×	×

28. Production Impact

Application Type	Usefulness
Framework/library inheritance	? High
Domain model with inheritance	? Good
Complex constructor arguments	? High
Simple POJO	? Low
DTOs	? Low
Spring Boot service classes	? Usually low
Configuration classes	? Sometimes
Numeric/data transformation classes	? Good
Legacy inheritance-heavy code	? Potentially useful
Builder-heavy domain model	? Depends
Public Java library	? Preview concern

General business logic	🔗 Low
-------------------------------	-------

29. Java 24 Preview Usage

Because JEP 492 is a preview feature, Java 24 requires preview support.

Compile:

```
javac --enable-preview --release 24 MyClass.java
```

Run:

```
java --enable-preview MyClass
```

This means it should be adopted carefully in production systems where long-term source compatibility is important.

30. Final Recommendation

Scenario	Recommendation
Need simple super(value)	🔗 Traditional constructor
Need to normalize argument before super()	🔗 Flexible Constructor Bodies
Need validation before super()	🔗 Flexible Constructor Bodies
Need multiple calculations before super()	🔗 Flexible Constructor Bodies
Need parsing before super()	🔗 Flexible Constructor Bodies
Need defensive copy before super()	🔗 Flexible Constructor Bodies
Need complex creation workflow	🔗 Static Factory
Many optional constructor parameters	🔗 Builder
Reusable transformation	🔗 Static helper
Database/network operation	🔗 Don't put it before super()
Need this before super()	🔗 Not allowed

Stable production API	⚠ Be cautious because it is preview

Bottom line

Flexible Constructor Bodies (Third Preview) removes the artificial restriction that `super(...)` or `this(...)` must always be the first executable statement in a constructor.

Its ideal pattern is:

```

Constructor
↓
Validate / normalize / calculate
↓
super(...) or this(...)
↓
Initialize current object

```

For example:

```

class Employee extends Person {

    Employee(String name, double salary) {

        String normalizedName = name.trim();
        double normalizedSalary = Math.max(0, salary);

        super(normalizedName);

        // initialize Employee-specific state
    }
}

```

The feature is most valuable when **a subclass needs to prepare constructor arguments before invoking its superclass constructor**. It should **not** be used to turn constructors into complex workflows. For substantial object creation logic, **static factory methods or builders remain better alternatives**.

One important version-tracking point: **JEP 492 was still a preview in Java 24**, so Java 24 code using this feature requires preview support; it should not be treated as a stable Java SE feature merely because it had already reached its third preview.

Java 24 — Linking Run-Time Images without JMODs

Property	Details
Java Version	24
Year	2025
Feature	Linking Run-Time Images without JMODs
JEP	JEP 493
UniqueClassName	LinkRuntimeImages
Status	Final
Main tool affected	jlink
Main idea	Allow jlink to create a custom runtime image from an existing runtime image (lib/modules) , without requiring the JDK's jmods directory
Primary benefit	A JDK distribution can omit JMOD files while still allowing custom runtime images to be linked
Main motivation	Reduce JDK installation size and make custom runtime generation possible from a smaller JDK distribution

JEP 493 was delivered in Java 24. It allows a specially built JDK with a **linkable runtime image** to serve as the base for jlink, rather than requiring packaged JMOD files. ([OpenJDK Bug System](#))

1. What Problem Does It Solve?

Before Java 24, jlink normally relied on JMOD files from:

`$JAVA_HOME/jmods/`

For example:

```
jmods/
├── java.base.jmod
├── java.desktop.jmod
├── java.logging.jmod
├── java.sql.jmod
└── ...
```

A typical command was:

```
jlink \  
--module-path $JAVA_HOME/jmods \  
--add-modules java.base,java.logging,java.sql \  
--output custom-runtime
```

The problem is that JMODs contain information needed for **linking**, but they duplicate much of the content already represented in the runtime image.

JEP 493 allows a specially built JDK to use its existing runtime image:

```
JDK  
├── bin/  
├── conf/  
├── lib/  
│   └── modules ← runtime image  
└── ...
```

instead of requiring:

```
JDK  
├── jmods/  
│   ├── java.base.jmod  
│   ├── java.sql.jmod  
│   └── ...  
└── lib/modules
```

The OpenJDK issue describing JEP 493 notes that packaged modules can add roughly **80–90 MB** to a JDK installation and are needed primarily for linking custom runtime images, not normal runtime execution. ([OpenJDK Bug System](#))

2. Traditional Architecture vs Java 24

Aspect	Traditional JDK	JDK 24 with JEP 493
Runtime classes	lib/modules	lib/modules
JMOD files	Required by traditional jlink workflow	Not necessarily required
jmods/ directory	Normally present for linking	Can be absent

Custom runtime generation	Requires JMODs	Can use linkable runtime
JDK installation size	Larger	Potentially smaller
Primary tool	jlink	jlink
Runtime behavior	Normal	Normal
Main change	Link from JMODs	Link from runtime image

3. What Exactly Is a JMOD?

A JMOD is a packaging format used primarily for **link-time** and certain development purposes.

For example:

```
java.base.jmod
java.sql.jmod
java.desktop.jmod
```

Oracle's documentation describes JMOD as a format that can contain not only classes and resources but also native libraries and other configuration files. JMOD files are **not used at run time**; they are intended for compile/link-time scenarios. ([Oracle Docs](#))

So conceptually:

```
JMOD
↓
Linking
↓
Custom runtime image
↓
Application runs from runtime image
```

JEP 493 reduces the need to ship those JMODs merely so that jlink can create another runtime.

4. How jlink Fits Into This

jlink assembles modules and their transitive dependencies into a custom runtime image. ([Oracle Docs](#))

For example:

```
jlink \  
--add-modules java.base,java.logging,java.sql \  
--output my-runtime
```

The resulting runtime might look conceptually like:

```
my-runtime/  
├── bin/  
│   └── java  
├── conf/  
├── lib/  
│   └── modules  
└── ...
```

Then:

```
my-runtime/bin/java -version
```

can run the application without requiring a full JDK installation.

5. Advantages

Advantage	Explanation
No JMOD requirement for linkable JDKs	jlink can use the runtime image itself as the basis for linking
Smaller JDK distributions	JMOD files can be omitted from specially built JDK distributions
Reduces duplicated content	Runtime image and JMODs otherwise contain overlapping information
Useful for containers	Smaller build/runtime images can reduce container footprint
Better minimal-Java deployments	Useful when a JDK is distributed specifically for application deployment
Simplifies some Linux distributions	Package maintainers can omit JMODs while retaining runtime-linking capability
Supports custom runtime images	Applications can still obtain trimmed Java runtimes
Good for cloud	Smaller artifacts can reduce image/storage transfer costs

deployments	
Works with existing jlink concept	Developers still create custom runtime images rather than changing application architecture
Preserves modular deployment model	Java modules remain the unit used for runtime-image construction
Potentially better distribution flexibility	Vendors can provide JDK variants optimized for different purposes

The motivation is particularly strong for distributions that don't want to install the additional JMOD package but still want custom runtime-image creation. ([OpenJDK Bug System](#))

6. Disadvantages

Disadvantage	Explanation
Not every JDK is automatically linkable	A JDK must be built/configured appropriately
Special build configuration	Linkable runtime support is enabled at JDK build time
Not a general application feature	It mainly affects JDK/runtime-image tooling
Doesn't eliminate modularity requirements	jlink still works with modules and dependencies
Potential tool compatibility considerations	Build and packaging pipelines may need adjustment
Existing scripts may assume jmods/ exists	CI/CD scripts can break if they explicitly reference JMOD paths
Third-party tooling may expect JMODs	Some build systems or tools may still rely on traditional JMOD layouts
Limited value for ordinary Java applications	Applications that simply run on a standard JDK gain little directly
Platform-specific runtime images remain necessary	A runtime image built for Linux isn't a universal Windows runtime
JDK maintainers carry more configuration choices	Vendors must decide whether to ship JMODs, a linkable runtime, or both
Some target-runtime restrictions exist	A linkable runtime has limitations around using

	jdk.jlink in target images
--	----------------------------

A notable consequence documented in OpenJDK issue tracking is that a JEP 493-enabled JDK cannot simply be used to create another image containing the jdk.jlink module in the same way as a conventional JDK with packaged modules. ([OpenJDK Bug System](#))

7. When to Use

Situation	Recommendation	Reason
Building a custom JDK distribution	👍 Excellent	JMODs may be omitted
Containerized Java applications	👍 Excellent	Smaller distributions can be useful
Cloud deployment	👍 Good	Reduces artifact footprint
Linux package distribution	👍 Excellent	JMODs can be separated/omitted
Embedded Java deployment	👍 Excellent	Smaller runtime/tool footprint
Creating application-specific runtimes	👍 Good	jlink remains useful
CI/CD generating custom runtimes	👍 Good	Can work without traditional JMOD packages
Desktop application packaging	👍 Good	Custom runtime can be bundled with application
Microservices	👍 Good	Each service can have a minimal runtime
Standard developer workstation	👍 Moderate	Full JDK with JMODs may still be more convenient
Simple Java application	👍 Low	Usually no need to build a custom runtime

8. When NOT to Use

Situation	Recommendation	Why
You simply need to run a Java application	❓ Don't bother	Install/use a normal JRE/JDK
You need standard Java development tools	❓ Prefer normal JDK	JMOD-less distributions may have limitations
Your build requires JMOD files	❓ Keep a JDK with JMODs	Required by that workflow
Third-party tooling expects \$JAVA_HOME/jmods	❓ Use conventional JDK	Avoid compatibility problems
You aren't creating custom runtime images	❓ Little benefit	Feature is primarily about linking
Cross-platform packaging	❓ Carefully plan	Runtime images are platform-specific
You need maximum JDK tooling compatibility	❓ Conventional JDK	Traditional JDK is simpler
Team has no modular application	❓ Usually unnecessary	jlink itself may not provide enough value

9. Real-Time Use Case — Microservice Container

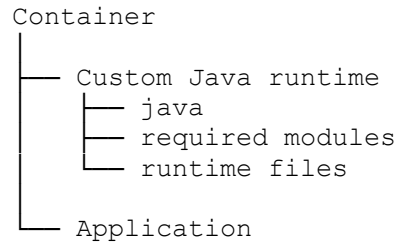
Imagine a Spring Boot or modular Java microservice.

Traditional deployment:

Container

- Full JDK
- jmods/
- Development tools
- Documentation
- Application
- Configuration

With a custom runtime:



The idea is:

```
jlink \
--add-modules java.base,java.logging,java.sql \
--strip-debug \
--compress=2 \
--output runtime
```

Then package:

```
runtime/
application.jar
configuration/
```

Oracle documents jlink specifically for creating custom runtime images and provides plugins such as `--strip-debug` and `--compress` to reduce the resulting image. ([Oracle Docs](#))

10. Real-Time Use Case — Docker

A company might have:

```
Application
↓
Java modules
↓
jlink
↓
Custom runtime
↓
Docker image
```

Instead of putting a complete JDK inside the final container:

```
COPY custom-runtime /opt/java
COPY application.jar /app/
```

```
ENTRYPOINT ["/opt/java/bin/java", "-jar", "/app/application.jar"]
```

This is especially attractive when the organization already has a standardized process for producing custom runtime images.

11. Real-Time Use Case — Desktop Application

Consider an enterprise desktop application:

```
AccountingApplication/  
├── accounting.exe  
├── app/  
│   └── accounting.jar  
└── runtime/  
    ├── bin/  
    └── lib/
```

The user doesn't need to install Java separately.

jpackage can package an application with a custom runtime. Oracle documents using jlink to create a custom runtime and then passing it to jpackage with `--runtime-image`. ([Oracle Docs](#))

Conceptually:

```
jlink \  
--add-modules java.base,java.desktop,java.logging \  
--output custom-runtime
```

Then:

```
jpackage \  
--name AccountingApp \  
--runtime-image custom-runtime \  
...
```

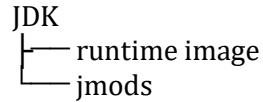
12. Real-Time Use Case — Linux Distribution

Suppose a Linux distribution wants:

java-runtime

to be small.

Historically:

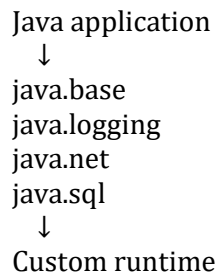


The distribution might package JMODs separately because they are mainly useful for linking custom runtime images.

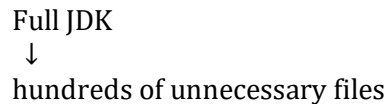
JEP 493 makes it possible to provide a JDK/runtime distribution where the runtime itself can be used for linking without requiring a separate JMOD package. This directly addresses the packaging issue described by OpenJDK. ([OpenJDK Bug System](#))

13. Real-Time Use Case — Embedded Java

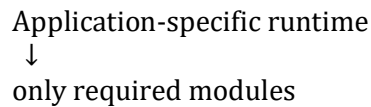
Consider a device running:



Instead of deploying the entire JDK:



the organization can produce:



This can be particularly valuable when storage or update bandwidth is constrained.

14. Real-Time Use Case — CI/CD

A build pipeline could be:



Compile
↓
Test
↓
jlink
↓
Custom Runtime
↓
Application Package
↓
Container / Installer

For example:

```
jlink \  
  --add-modules java.base,java.logging,java.sql \  
  --strip-debug \  
  --compress=2 \  
  --output build/runtime
```

Then:

```
jpackage \  
  --name MyApplication \  
  --runtime-image build/runtime \  
  ...
```

This produces a deployment artifact containing the runtime needed by the application. Oracle's Java 24 packaging documentation explicitly describes this jlink → jpackage workflow. ([Oracle Docs](#))

15. JMOD vs Runtime Image

Characteristic	JMOD	Runtime Image
Main purpose	Link-time packaging	Execution
Typical location	jmods/*.jmod	lib/modules
Executable directly?	✗ No	✓ Yes, as part of runtime
Used by traditional jlink	✓	Indirectly
Used at normal application runtime	✗	✓
Contains module information	✓	✓

Can contain native libraries	✓	Runtime contains required native components
Needed with JEP 493 linkable runtime	Not necessarily	✓
Best role	Development/linking	Deployment/execution

Oracle specifically states that JMOD files are not executable and are intended for compile/link-time use. ([Oracle Docs](#))

16. JEP 493 vs Traditional jlink

Aspect	Traditional jlink	JEP 493
Input	JMODs / modules	Runtime image can be used
jmods/	Normally required for JDK modules	Can be absent
JDK size	Larger	Potentially smaller
Custom runtime	✓	✓
Main benefit	Established workflow	Reduced JDK distribution footprint
Configuration	Standard	Linkable runtime build required
Application code changes	None	None
Deployment benefit	Custom runtime	Custom runtime + smaller tooling distribution

17. JEP 493 vs jpackage

These features are related but solve different problems.

Feature	JEP 493	jpackage
Primary purpose	Enable runtime linking	Package Java applications

	without JMODs	
Creates runtime	Supports jlink creation	Can invoke jlink
Creates installer	✗	✓
Creates app image	✗	✓
Windows MSI	✗	✓
macOS package	✗	✓
Linux packages	✗	✓
Main concern	JDK/runtime-image construction	Application distribution

jpackage is the **packaging layer**; JEP 493 is a **runtime-image linking improvement**.

18. JEP 493 vs Classpath/JAR Deployment

Aspect	JAR/ClassPath	JEP 493 + jlink
Requires modular application	✗	Usually beneficial with modules
Custom runtime	✗	✓
Full JDK required at deployment	Usually no, but runtime required	Custom runtime bundled
Runtime size optimization	Limited	Strong
Module dependency analysis	✗	✓
Deployment complexity	Low	Higher
Best for	Simple applications	Controlled enterprise deployment

19. Alternate Features / Approaches

There is **no exact replacement** for JEP 493 because it changes how jlink obtains the JDK modules. However, several alternatives solve the broader problem of creating or distributing a smaller Java runtime.

Alternative	Purpose	When Better
Traditional jlink + JMODs	Create custom runtime	You have a normal JDK with jmods/
package	Package application + runtime	Desktop/application distribution
Container base images	Provide minimal Java runtime	Cloud/container deployment
Standard JRE/runtime image	Run applications	No custom module trimming required
Custom JDK distribution	Control JDK contents	Organizations maintaining their own Java platform
GraalVM Native Image	Compile application to native executable	When native executable/startup footprint is the goal
Modular JAR + normal runtime	Simplify deployment	When custom runtime size isn't important
CDS/AppCDS	Improve startup/memory behavior	When startup/runtime efficiency matters more than runtime size

20. JEP 493 vs GraalVM Native Image

These are sometimes confused because both can reduce deployment footprints, but they work very differently.

Aspect	JEP 493 + jlink	Native Image
Output	Java runtime image	Native executable
JVM required	✓	✗ at application runtime
Java bytecode	Retained	Compiled ahead of time
JVM JIT	✓	✗

Java compatibility	Very high	Requires compatibility analysis
Startup	Good	Often very fast
Runtime optimization	JIT	AOT
Dynamic features	Strong	More constrained
Build complexity	Low/medium	Medium/high
Best for	Custom Java runtime	Native deployment

21. Performance Considerations

An important point:

JEP 493 is primarily a packaging and deployment improvement, not a JVM execution-performance feature.

Area	Expected Impact
Application throughput	Usually no direct improvement
JVM execution	Essentially unchanged
Startup	Depends on resulting runtime and configuration
Disk footprint	🔍 Potential improvement
Container size	🔍 Potential improvement
Download size	🔍 Potential improvement
Deployment efficiency	🔍 Potential improvement
Memory usage	Can improve indirectly by excluding unnecessary components, but workload-dependent
Build complexity	🔍 Slightly more specialized
Application source code	No change required

Don't choose JEP 493 because you expect your Java code to execute faster.

Choose it because you want a **more efficient Java distribution/runtime-image creation workflow**.

22. Important Limitation — Linkable Runtime

JEP 493 is not simply:

Install any Java 24 runtime



jlink works without JMODs

The JDK must be built/configured to support this mode.

OpenJDK's implementation describes a build-time option:

--enable-linkable-runtime

for enabling the linkable-runtime behavior. ([OpenJDK Bug System](#))

Therefore:

JDK Type	JMOD-free linking
Conventional JDK with jmods/	✗ Not the JEP 493 scenario
JEP 493-enabled/linkable JDK	✓
Ordinary JRE/runtime image	✗ Generally not
Custom vendor JDK	Depends on vendor build
Application's custom jlink image	Depends on how it was produced

23. Best Practices

Practice	Recommendation
Use modular applications	🔍 Recommended
Analyze dependencies before linking	🔍 Recommended

Use jlink for custom runtime creation	☑ Recommended
Use --strip-debug when appropriate	☑ Good
Use compression where appropriate	☑ Good
Test the resulting runtime thoroughly	☑ Essential
Keep the JDK used for building documented	☑ Essential
Don't assume every JDK has linkable-runtime support	☑ Important
Don't assume JEP 493 improves application performance	☑ Important
Use jpackage when an installer is required	☑ Recommended
Keep a conventional JDK for development if needed	☑ Practical

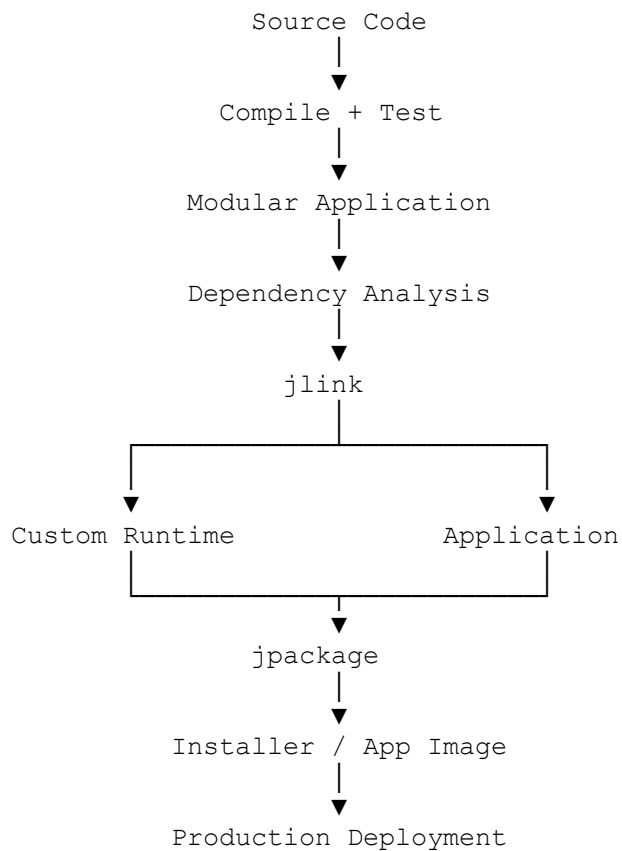
24. Decision Table

Question	Recommendation
Do you need a custom Java runtime?	☑ Consider jlink
Do you want to remove the JMOD requirement?	☑ JEP 493
Are you building a small JDK distribution?	☑ Strong candidate
Are you deploying containers?	☑ Strong candidate
Are you building embedded Java systems?	☑ Strong candidate
Are you building desktop installers?	☑ JEP 493 + jpackage can be useful
Do you only need to execute a JAR?	☑ Probably unnecessary
Do your tools require \$JAVA_HOME/jmods?	☑ Keep a conventional JDK

Do you need a native executable?	❓ Consider Native Image instead
Do you need an installer?	❓ Use jpackage
Do you need maximum JDK tooling compatibility?	❓ Conventional JDK may be better

25. Enterprise Real-Time Architecture

A modern enterprise deployment can look like:



This is especially useful for organizations that want to control exactly which Java modules reach production.

26. Overall Advantages vs Disadvantages

Category	Assessment
JDK distribution size	☑ Excellent
Custom runtime creation	☑ Excellent
Container deployment	☑ Very useful
Embedded deployment	☑ Very useful
Desktop packaging	☑ Useful with jpackage
Developer convenience	☑ Moderate
Application performance	☑ Little direct effect
Compatibility	☑ Requires planning
Build complexity	☑ Moderate
Migration complexity	☑ Low if already using jlink
Benefit for simple JAR applications	☑ Low
Enterprise deployment	☑ High

27. Final Recommendation

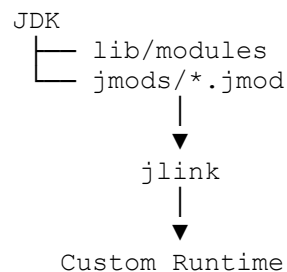
Scenario	Best Choice
Full JDK development	Conventional JDK
Custom small runtime	JEP 493 + jlink
Docker/microservice	JEP 493 + jlink + container
Desktop application	jlink + jpackage
Embedded Java	JEP 493 + jlink
Need only a JAR runtime	Normal Java runtime
Need native executable	GraalVM Native Image / equivalent AOT

	approach
Need JMOD-dependent tooling	Conventional JDK with JMODs
Need installer generation	Jpackage

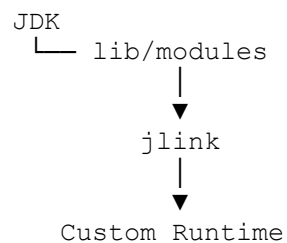
Bottom line

JEP 493 is primarily a Java distribution and deployment optimization.

The traditional model is:



Java 24 introduces the ability for a suitably built JDK to use the runtime image itself:



So the biggest practical benefit is:

You can distribute a JDK without JMODs yet still retain the ability to create custom runtime images, provided the JDK was built as a linkable runtime.

This is particularly valuable for **cloud/container platforms, Linux distributions, embedded systems, and organizations producing custom Java runtimes**. It is much less important for ordinary developers who simply compile and run Java applications. ([OpenJDK Bug System](#))

Java 24 — Module Import Declarations (Second Preview)

Property	Details
Java Version	24
Year	2025
Feature	Module Import Declarations (Second Preview)
JEP	494
UniqueClassName	ModuleImportDeclarations
Status	Second Preview
First Preview	Java 23
Primary goal	Import all accessible top-level classes and interfaces exported by a module
Main syntax	import module java.sql;
Key benefit	Reduces many individual import statements when using an entire module
Related features	Ordinary imports, wildcard imports, automatic imports from java.lang, static imports
Important	This is a language feature , not a runtime-performance feature

The feature was introduced as JEP 476 in Java 23 and refined as **JEP 494** for Java 24. It allows a declaration such as `import module java.sql;` to make the public top-level classes/interfaces exported by that module available by simple name.

1. What Are Module Import Declarations?

Normally, if you use several classes from a module, you write individual imports:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
```

Or a package wildcard:

```
import java.sql.*;
```

Module Import Declarations allow:

```
import module java.sql;
```

Conceptually:

Module

↓

java.sql

↓

Accessible exported types

↓

Available for simple-name use

For example:

```
import module java.sql;
```

```
class DatabaseExample {  
    void execute() throws SQLException {  
        Connection connection =  
            DriverManager.getConnection(  
                "jdbc:mysql://localhost/test"  
            );  
        PreparedStatement statement =  
            connection.prepareStatement(  
                "SELECT * FROM employee"  
            );  
        ResultSet resultSet =  
            statement.executeQuery();  
    }  
}
```

Instead of importing each java.sql type individually.

2. Module Import vs Package Wildcard

This distinction is extremely important.

Traditional package wildcard

```
import java.sql.*;
```

means:

Import accessible types from **one package**.

Module import

```
import module java.sql;
```

means:

Import accessible top-level types exported by **the entire module**, potentially across multiple packages.

Feature	import java.sql.*	import module java.sql
Scope	One package	Entire module
Multiple packages	✗	✓
Module-aware	✗	✓
Reduces imports	Somewhat	More significantly
Requires JPMS module knowledge	✗	✓
Useful for modular applications	Good	Excellent
Syntax complexity	Very low	Low

3. Why Was This Feature Introduced?

Consider a modular application using several packages from one module.

For example, a module could export:

```
com.example.database  
com.example.database.model  
com.example.database.util  
com.example.database.query
```

Without module imports:

```
import com.example.database.Connection;
import com.example.database.model.Customer;
import com.example.database.model.Order;
import com.example.database.query.Query;
import com.example.database.util.DatabaseUtils;
```

With:

```
import module com.example.database;
```

the compiler can make the accessible exported types available.

This is particularly attractive for **large APIs split across multiple packages**.

4. Basic Example

Without Module Import

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
```

With Module Import

```
import module java.net.http;
```

Then:

```
class HttpExample {

    void call() throws Exception {

        HttpClient client =
            HttpClient.newHttpClient();

        HttpRequest request =
            HttpRequest.newBuilder(
                URI.create("https://example.com")
            ).build();

        HttpResponse<String> response =
            client.send(
                request,
                HttpResponse.BodyHandlers.ofString()
            );
    }
}
```

}

The module import can eliminate several explicit imports from the same module.

5. Advantages

Advantage	Explanation
Fewer import statements	One declaration can replace many individual imports
Works across packages	Unlike <code>package.*</code> , it applies at module level
Excellent for large APIs	Useful when an API spans several packages
Module-aware	Aligns imports with the Java Platform Module System
Improves readability	Makes it obvious that the code depends on a module
Reduces import maintenance	Adding another type from the same module doesn't require another import
Useful for exploratory code	Convenient when using many types from a module
Good for teaching/demos	Less import boilerplate
Works with standard modules	Can simplify use of modules such as <code>java.sql</code>
Can simplify generated code	Code generators can emit fewer import declarations
Supports modular API design	Encourages thinking in terms of module boundaries

6. Disadvantages

Disadvantage	Explanation
Preview feature	In Java 24 it is still a preview
Requires preview compilation	Java 24 code needs preview enabled

Less explicit dependencies	Individual imports show exactly which classes are used
Potential name conflicts	Multiple modules may expose types with identical simple names
Requires JPMS understanding	Developers need to understand exports/readability
Can import more names than expected	Module-level scope is broader than package wildcard
IDE/tooling adoption	Some tooling may initially handle preview syntax differently
Can hide API boundaries at type level	You see the module but not the specific imported types
Not necessarily useful for small APIs	One or two explicit imports may be clearer
May reduce conventional Java familiarity	Developers are more accustomed to normal imports
Doesn't improve runtime performance	It mainly changes source-code ergonomics

7. Important: It Does NOT Mean "Import Everything in the Module"

The declaration doesn't blindly make every class in the module accessible.

The relevant types must satisfy Java's accessibility/module rules.

Conceptually:

```
import module X
├── exported type A → available
├── exported type B → available
├── exported type C → available
└── non-exported type → not available
```

So module encapsulation is still respected.

This is one of the important differences from simply treating the module as an unrestricted collection of classes.

8. Advantages in Enterprise Applications

Enterprise scenario	Benefit
Large modular application	Fewer imports
Internal platform modules	Easy access to related APIs
Shared libraries	Simplifies client source
Multi-package API	Better than many package imports
Code generation	Less generated boilerplate
Prototyping	Faster development
Educational projects	Easier examples
API-heavy code	Cleaner source
JPMS-based architecture	Natural module-level dependency expression

9. When to Use

Situation	Recommendation	Reason
Using many classes from one module	👍 Excellent	Reduces import declarations
API spans multiple packages	👍 Excellent	Package wildcard cannot cover multiple packages
JPMS modular application	👍 Excellent	Module-level dependency is natural
Java platform module with many types	👍 Good	Less boilerplate
Generated source code	👍 Good	Fewer generated imports
Educational/demo code	👍 Good	Simpler examples
Prototype	👍 Good	Convenience

Stable production code	☒ Evaluate	Java 24 feature is preview
Only one or two imported types	☒ Usually unnecessary	Explicit imports are clearer
API has many name collisions	☒ Avoid	Explicit imports provide more control

10. When NOT to Use

Situation	Why Avoid It?	Better Approach
Only one type is needed	Module import adds unnecessary breadth	Explicit import
Only two types are needed	Explicit imports are clearer	Explicit import
Multiple modules expose same type name	Name ambiguity	Explicit imports
Team avoids preview features	Java 24 preview dependency	Standard imports
Public library requiring stable source	Preview is undesirable	Standard imports
Very strict dependency visibility	Explicit imports communicate usage better	Explicit imports
Code review emphasizes precise dependencies	Wildcard/module imports obscure details	Explicit imports
Legacy project with no JPMS	Little benefit	Normal imports
API has frequent naming collisions	Can cause ambiguity	Explicit imports

11. Real-Time Use Case — JDBC Application

Suppose an application uses:

Connection
DriverManager
PreparedStatement
ResultSet
SQLException
Statement

Traditional

```
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.PreparedStatement;  
import java.sql.ResultSet;  
import java.sql.SQLException;  
import java.sql.Statement;
```

Module import

```
import module java.sql;
```

Then:

```
class EmployeeRepository {  
  
    List<String> findNames(Connection connection)  
        throws SQLException {  
  
        PreparedStatement ps =  
            connection.prepareStatement(  
                "SELECT name FROM employee"  
            );  
  
        ResultSet rs = ps.executeQuery();  
  
        // ...  
        return List.of();  
    }  
}
```

Benefit

6+ import declarations
↓
1 module import

This is a very natural use case.

12. Real-Time Use Case — HTTP Client

Traditional:

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
```

Module import:

```
import module java.net.http;
```

Then:

```
class PaymentClient {

    void send() throws Exception {

        HttpClient client =
            HttpClient.newHttpClient();

        HttpRequest request =
            HttpRequest.newBuilder(
                URI.create("https://payment.example")
            ).build();

        HttpResponse<String> response =
            client.send(
                request,
                HttpResponse.BodyHandlers.ofString()
            );
    }
}
```

This is useful in HTTP-heavy applications.

13. Real-Time Use Case — Internal Enterprise Module

Suppose your organization has:

```
com.company.platform
├── com.company.platform.auth
├── com.company.platform.logging
├── com.company.platform.config
├── com.company.platform.metrics
└── com.company.platform.security
```

and exports these packages.

Traditional:

```
import com.company.platform.auth.User;
import com.company.platform.auth.Role;
import com.company.platform.logging.Logger;
import com.company.platform.metrics.Metrics;
import com.company.platform.config.Configuration;
```

Module import:

```
import module com.company.platform;
```

This becomes attractive when the application genuinely uses a large portion of the platform module.

14. Real-Time Use Case — Generated Java Code

Suppose a code generator produces:

```
import com.company.api.Customer;
import com.company.api.Order;
import com.company.api.Product;
import com.company.api.Invoice;
import com.company.api.Payment;
```

If these classes are spread across exported packages in one module:

```
import module com.company.api;
```

The generator can produce much less boilerplate.

Concern	Traditional	Module Import
Generated import lines	Many	Few
Generator complexity	Higher	Lower
Maintenance	More	Less
Name conflicts	Easier to detect explicitly	Need handling

15. Real-Time Use Case — Data Processing

Imagine a modular analytics library:

```
com.company.analytics
├── model
├── query
├── aggregation
├── statistics
└── visualization
```

An analytics application may need:

Dataset
Query
Filter
Aggregation
Statistics
Chart

Instead of:

```
import com.company.analytics.model.Dataset;
import com.company.analytics.query.Query;
import com.company.analytics.query.Filter;
import com.company.analytics.aggregation.Aggregation;
import com.company.analytics.statistics.Statistics;
import com.company.analytics.visualization.Chart;
```

you could use:

```
import module com.company.analytics;
```

This is particularly useful when the module represents a cohesive API.

16. Name Conflict Problem

Consider two modules:

```
module com.company.customer
  exports com.company.customer;
```

```
module com.company.billing
  exports com.company.billing;
```

Both might have:

Customer

If you import both modules:

```
import module com.company.customer;  
import module com.company.billing;
```

you could have ambiguity when writing:

Customer customer;

In such cases, explicit imports or fully qualified names may be preferable.

17. Module Import vs Explicit Import

Aspect	Explicit Import	Module Import
import java.sql.Connection	✓	✗ unnecessary
Import entire module	✗	✓
Exact dependency visible	★★★★★	★★★
Boilerplate	★★	★★★★★
Name-conflict control	★★★★★	★★★
Large API convenience	★★★	★★★★★
Familiarity	★★★★★	★★★
Preview in Java 24	✗	?
Best for stable library APIs	★★★★★	★★★

18. Module Import vs Package Wildcard

Aspect	Explicit	Package *	Module Import
import x.Y	✓	✗	✗

import x.*	✗	✓	✗
import module x	✗	✗	✓
Scope	Type	Package	Module
Multiple packages	✗	✗	✓
Name ambiguity potential	Low	Medium	Higher
Boilerplate	High	Low	Very low
Dependency precision	Highest	Medium	Lower
JPMS-aware	No	No	Yes

19. Module Import vs Static Import

These solve completely different problems.

Feature	Purpose	Example
Explicit import	Import one type	import java.util.List;
Wildcard import	Import package types	import java.util.*;
Module import	Import types exported by module	import module java.sql;
Static import	Import static members	import static java.lang.Math.PI;
Static wildcard	Import all static members	import static java.lang.Math.*;

Therefore:

import module java.sql;

does **not** replace static imports.

20. Alternate Features

There isn't one exact replacement, but the alternatives are straightforward.

Alternative	Purpose	Best When
Explicit type imports	Import exact classes	Production code requiring maximum clarity
Package wildcard imports	Import package types	Many types from one package
Static imports	Import static members	Utility-heavy code
Fully qualified names	Avoid ambiguity	Name collisions
IDE automatic imports	Manage imports automatically	Normal development
Refactoring/package organization	Reduce import complexity	API design problems
Facade API	Expose a smaller API surface	Enterprise library design

21. Which Alternative Should You Choose?

Situation	Best Alternative
1-3 classes from module	Explicit imports
Many classes from one package	Package wildcard
Many classes across one module	Module import
Same type name from multiple modules	Explicit imports
Static utility methods	Static imports
Need maximum source clarity	Explicit imports
Need stable Java syntax	Explicit/package imports
Large internal modular platform	Module import
Public API/library	Usually explicit imports

22. Impact on Performance

Area	Impact
Runtime performance	✗ No meaningful direct benefit
Startup time	✗ Not a performance feature
Memory consumption	✗ No significant direct benefit
Compilation	🔍 Compiler resolves imports; workload/tool dependent
Source-code size	🔍 Smaller
Developer productivity	🔍 Potentially better
Maintainability	🔍 Potentially better for large modules
Application architecture	🔍 Encourages module-level thinking

Important: Module Import Declarations are fundamentally a **source-code convenience feature**.

They do not mean:

module import
↓
faster application

Instead:

module import
↓
less import boilerplate
↓
potentially cleaner source

23. Best Practices

Practice	Recommendation
Use for APIs spanning many packages	🔍

Use with JPMS applications	?
Use when many types from one module are required	?
Keep module boundaries well designed	?
Use explicit imports for ambiguous types	?
Don't use merely to reduce 1-2 imports	?
Avoid unnecessary module-wide imports	?
Consider team coding standards	?
Test IDE/build-tool compatibility	?
Remember Java 24 preview status	?

24. Production Decision Table

Project Type	Recommendation
Small Java application	? Usually unnecessary
Large JPMS application	? Good candidate
Enterprise modular platform	? Excellent candidate
Internal shared modules	? Good
Public Java library	? Use cautiously
Educational project	? Good
Prototype	? Good
Generated source	? Good
API with many packages	? Excellent
API with frequent name collisions	? Prefer explicit imports
Java 24 production application avoiding previews	? Don't use

Modern codebase embracing Java previews	❓ Consider it
---	---------------

25. Overall Comparison

Criterion	Explicit Import	Package Wildcard	Module Import
Simplicity	☆☆☆	☆☆☆☆	☆☆☆☆☆
Precision	☆☆☆☆☆	☆☆☆	☆☆
Boilerplate reduction	☆☆	☆☆☆☆	☆☆☆☆☆
Multi-package support	✗	✗	✓
Name conflict avoidance	☆☆☆☆☆	☆☆☆	☆☆
JPMS integration	☆☆☆	☆☆☆	☆☆☆☆☆
Familiarity	☆☆☆☆☆	☆☆☆☆☆	☆☆☆
Large API usability	☆☆☆	☆☆☆☆	☆☆☆☆☆
Stable Java 24 code	☆☆☆☆☆	☆☆☆☆☆	❓ Preview
Best general-purpose choice	❓	❓	❓

26. Final Recommendation

Scenario	Best Choice
Few classes	Explicit imports
Many classes in one package	Package wildcard
Many classes across one module	Module Import Declaration
JPMS-heavy enterprise application	Module Import Declaration

Name conflicts	Explicit imports
Public library	Explicit imports
Generated source	Module Import Declaration can be useful
Java 24 preview-free project	Explicit/package imports
Learning/demo code	Module Import Declaration can reduce noise

Bottom line

Module Import Declarations (Second Preview) are primarily a readability and productivity improvement for modular Java applications.

The key difference is:

Explicit import



one type

Package wildcard



one package

Module import



accessible exported types
from an entire module

For example:

```
import module java.sql;
```

can replace a collection of imports such as:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
```

Use it when your code consumes many types from a cohesive module, especially a large JPMS-based API. Avoid it when only a few types are needed, when name conflicts are common, or when your project avoids preview features.

And one practical caution: because **Java 24's JEP 494 is a second-preview feature**, it should be treated differently from finalized Java language features when designing a long-lived production codebase.